



INTERNATIONAL
HELLENIC
UNIVERSITY

[1]

Mining Greek Job Postings: Estimating Role Frequency and Regional Variation

Ioannis Kozas

SID: 3306230007

SCHOOL OF SCIENCE & TECHNOLOGY

A thesis submitted for the degree of

Master of Science (MSc) in Mobile and Web Computing:

IoT Applications

APRIL 2026

THESSALONIKI – GREECE



INTERNATIONAL
HELLENIC
UNIVERSITY

Mining Greek Job Postings: Estimating Role Frequency and Regional Variation

Ioannis Kozas

SID: 3306230007

Supervisor:

Prof. Leonidas Akritidis

Supervising Committee Members:

Assoc. Prof. Panagiotis Bozanis

Assist. Prof. Dimitrios Karapiperis

SCHOOL OF SCIENCE & TECHNOLOGY

A thesis submitted for the degree of

Master of Science (MSc) in Mobile and Web Computing:

IoT Applications

APRIL 2026

THESSALONIKI – GREECE

Abstract

The technologies, tools and products of the ICT sector are already diverse and evolve in an increasing rate aligning with the exponential development of the technology sector; the desired skills are adapting to that speed resulting in a frequent change of requirements and roles.

However, the methods of measuring those defining parameters that take place are not accessible to most employees, apart from the simple browsing resulting in a limited perspective of viewing one job post at a time taking place in a specific site. This perspective does not give the employee the overview of the parameters of the desired field in a defined region. In the available tools of the average employee which reside mainly to the job posting sites, the metrics and the real time statistics of a hard skill for example are not available, or the average requirements that a specific role has at a given time based on real-time data from the most used sites. It is not possible to make such queries with the available tools not only because they are not designed to do so, but also because the information is kept inside platforms, not giving the ability to make valuable conclusions from the centralisation of those data, hence results that can be useful for not only employees but for other stakeholders as well. This is a subject that have been researched many times, and the aims mentioned are technologically feasible as this thesis also confirms.

The centralisation of such data can be useful not only for individuals but can also serve as high level image of a sector, measuring not only skills and its demand on the market but the technologies that are used for the field that is leading innovation and eventually ‘upgrades’ the productivity of all the sectors. The focus of this thesis is on that aspect, the measurement of the core parameters the technology sector is using. I would like to express my sincere gratitude to Dr. Leonidas Akritides for supervising this thesis.

Ioannis Kozas

19/04/2026

Contents

ABSTRACT	III
CONTENTS	V
1 INTRODUCTION.....	7
2 LITERATURE REVIEW	10
2.1 RESEARCHING A SECTOR VIA THE JOB MARKET.....	10
2.2 TERMS EXPLAINED.....	12
2.3 RELATED WORK.....	13
3 EXTRACTION - WEB SCRAPING	20
3.1 WEB CRAWLING.....	20
3.1.1 <i>Web Crawler</i>	21
3.1.2 <i>Web Scraper</i>	21
3.1.3 <i>Web Spider</i>	21
3.2 SCRAPY	21
3.3 GENERAL OVERVIEW OF THE CRAWLING METHODS.....	22
3.3.1 <i>Configuration and Initialization of Scrapy</i>	22
3.3.2 <i>Scrapy Settings</i>	23
3.3.3 <i>Network Anonymization and Anti-Blocking Measures</i>	24
3.3.4 <i>Testing phase for locating the data within the DOM</i>	24
3.4 THE WEB CRAWLERS	26
3.4.1 <i>LinkedIn Crawling Issue</i>	28
4 DATA TRANSFORMATION	30
4.1 TRANSFORMATIONS WITHIN THE CRAWLERS.....	30
4.1.1 <i>Job ID Extraction and Cleaning</i>	30
4.1.2 <i>Description Field Cleaning</i>	31
4.1.3 <i>Concatenated Words Scraping Issue</i>	31
4.1.4 <i>The List Extraction Issue</i>	34

4.1.5	<i>Data Insertion to the Database</i>	35
5	DATABASE ARCHITECTURE	39
5.1	THE DATABASE.....	39
5.2	THE SCHEMA	39
5.3	DATA NORMALIZATION	43
5.3.1	<i>Regional Normalization</i>	44
5.3.2	<i>Regional Normalization in the Data Insertion Process</i>	46
5.3.3	<i>Date Format Normalization</i>	48
6	DATA ANALYSIS	49
6.1	PRE-PROCESSING AND DATA CLEANING	49
6.2	DATA ANALYSIS ENVIRONMENTS	50
6.2.1	<i>Spark in a Virtual Machine</i>	50
6.2.2	<i>Environment Isolation: Extraction vs. Data Analysis</i>	51
6.3	DATA ANALYSIS METHODOLOGIES	60
6.3.1	<i>TF-IDF Algorithm</i>	60
6.3.2	<i>Data Analysis using TF-IDF</i>	61
6.3.3	<i>Adapting SARO for Dictionary-Based Entity Extraction</i>	69
6.3.4	<i>Dictionary-based Entity Extraction Method Implementation</i>	70
6.3.5	<i>Analysis of The Roles Corresponding to Items</i>	79
7	CONCLUSIONS	89
	BIBLIOGRAPHY	92

1 Introduction

The goal of this thesis is to extract insights from a dataset that represents a small portion of the IT sector by using data analysis with Python, Pandas, Spark and Elasticsearch. The dataset was created by web scraping 4 sites: LinkedIn.com, Kariera.gr, Skywalker.gr and Jobfind.gr. The web scraping process was performed from September 2025 until April 2026, and the main part of the dataset is job posts from 10/2025 until 2026/02. It consists of 11 columns: id, job id, job title, job company, workplace, employment type, job sector, job date, job description, site, URL. Some of those columns had type conversions and corrections which mainly depend on the site from which the job posts crawled. The most important are conducted on the date and the workplace columns. The date field had a different date type for every site, so a date type conversion was developed for every site's spider¹. The workplace field type was different for every site and usually had differentiations even on the same site. Due to this I implemented a normalization method; I decided to work with cities as workplace and ignore more granular regional information. For example, the area of Patisia would be considered Athens, the area Neapoli would be considered Thessaloniki etc. There are other transformations but will be analyzed in later chapters.

The main method that was followed for the analysis was the “Dictionary-based Entity Extraction” Method which was inspired by Ontology-based Information Extraction (OBIE) and SARO Ontology [2] that was adapted for the implementation of this thesis. SARO Ontology consists of 'items' that have some specific basic properties; first is the id which is the name of the item. Second is the 'type', which is the general category of an item that can be either of the 3 following categories 'Product', 'Skill' and 'Tool'. Third is the 'labels' which is a list of words that identify as the item, essentially a list of strings that when one of them matches a string in the text that we apply regex algorithms, then this text relates with that SARO item and a connection is stored.

¹ The explanation of 'Spiders' is at the chapter 3.1.3 “Web Spider”

One SARO parameter was created for this research, which is the 'utility'. It was created to group specific items that this research was studying, for example the two basic groups that were investigated were 'Programming Languages' and 'Databases'. The way that it was used can be condensed into the following: use of SARO 'items' as tokens to match words in the text of the job description, when a word matches the id of the job posts of that job description is added into a list of ids.

After the end of the SARO process, each SARO item has a list of the job posts that this item appeared in. This list represents the presence of this item in the dataset, and its frequency can be used for comparison with other items either of the same category or even with items of different categories.

Apart from the main processing method of Dictionary-based Entity Extraction there are other methods and algorithms that investigate the dataset. This Thesis has these technically connected but distinct parts:

- A) Data Extraction and Storage on a PostgreSQL database
- B) Elasticsearch
- C) VirtualBox for Spark Processing
- D) Virtual Environment with Jupyter Notebook for Pandas and other Python libraries processing

The main jobs that are being done on each of the processing parts are the following:

Elasticsearch: Elasticsearch was first used for validation of data analysis algorithms and SQL results. The versatility of elasticsearch queries is what makes this tool very useful in investigating the content of the dataset. Another important use of Elasticsearch is the map feature, which is essentially the map placement of the dataset. By categorizing the job posts specific location into generalized locations or cities and then linking those aggregated locations to their geo-locations hence the latitude and longitude results into having all the specific job posts on an interactive map, which can be queried with specific values of the fields/columns or even string search to fields/columns for example the job description.

VirtualBox for Spark Processing: Here is where some basic NLP algorithms were used like Term Frequency and TF-IDF, those algorithms were applied in the fields of job description, site and job title, using the job description as the tokens input and the job title or the site as label input, those algorithms were operated with 2 or 4 labels and with

bigrams or one word as output. The results for the Term Frequency algorithm were simply the most frequent words used on specific kinds of job posts. The results for the TF-IDF were the most unique words that specific kind of job posts are using. For example, the TF-IDF results are giving the words ‘big data’, ‘mining’ and ‘data engineer’ high relativeness and uniqueness to the job title: ‘Data Engineer’.

Venv with Jupyter Notebook: Here is where the most results of this thesis were derived, the main method that I followed was: Utilizing the “Dictionary-based Entity Extraction”, using the SARO Ontology and further extending it by adding research related categories, and inserting the ‘utility’ category.

Data Extraction and Storing is the most crucial part and connects to the other three parts, Data Extraction and Storing consists of 3 main parts which are the following:

1. Web Scraping (Extract): The technique with which the data are collected from the job posting sites.
2. Transform: basic data cleaning and type correction on some fields like date.
3. Load: The data are inserted in the database (Postgres) after checking for duplicates.

Web scraping is being conducted with scrapy framework. [3] This is a free and open-source web-crawling framework written in Python. Generally, scrapy does two main jobs which are: first to conduct http requests to the pages that we want to crawl and second to only collect the valuable information from the pages. This is where the ‘spiders’ are used, which are customized algorithms that extract the specific data that we collect, also here is where a part of the ‘transform’ process is conducted, where html tags and spaces are removed, type conversion of dates and other processes that will be analyzed in detail later. The output of web scraping is JSON files which have the data crawled inside, organized into the respective fields of the database. This data though must be ‘cleaned’ from html tags, ‘\n’ symbols and spaces. For that job I developed a Python script that gets the data JSON as an input, cleans the job description and saves the data in the same JSON.

For LinkedIn an extra step had to be implemented, which is the extraction of job ids from the job posts for the next crawl. This was necessary because for the LinkedIn case the method of crawling was via a list of job ids which the spider iterated. The reason why this method was implemented and other technical details will be discussed later.

Loading data in PostgreSQL was conducted by python scripts which first check if the id of each job post in the JSON is already in the database; this is done to avoid duplicate values in the database. The main target of this Thesis was to extract valuable results from the dataset from which we can derive accurate conclusions about the dataset. As valuable are considered, results that depict reality best regarding the dataset in that case. As mentioned above the method that gave the most valuable results is the Dictionary-based Entity Extraction method, some conclusions that we can make for the Greek IT field from the results are the following: The first programming languages in use is Python for Backend and data related analysis, The second programming language in terms of demand is Java for backend, the first two databases in demand are Postgres and SQL Server.

2 Literature Review

This section presents a literature review that compares the findings of this research with those of existing studies on the same topic.

2.1 Researching a Sector via the Job Market

The research of a particular sector or even the job market, has been done in quite a lot of research and papers in the past years, especially with the development of big data where the possibilities of real-time data analysis have increased and as mentioned here [2] ‘reduces, if not erases the delay between the publishing of the job adverts and the moment it is available for analysis’.

The main research I relied on was the ‘**Ontology-guided Job Market Demand Analysis**’. This is the research where I get the SARO Ontology method and label job posts by using pre-defined criteria matching tokens of the job description. [4] This research paper introduces an automated framework for analyzing data science job market trends through advanced text processing. The Skills and Recruitment Ontology was developed to standardize the categorization of technical competencies, products, and tools found in online job postings. By employing Ontology-based Information Extraction (OBIE), the system identifies specific skill requirements with high accuracy compared to manual human indexing. These extracted data points are then processed using co-word analysis to visualize

the relationships and clusters between technologies. In this research paper they use that method to provide job seekers and educators with a strategic map of which skills are currently most interconnected and in high demand for tracking the evolution of digital labor market over time.

Below we can see the explanation of the SARO ontology from the paper “Ontology-guided Job Market Demand Analysis”

saro:JobPosting. – refers to a job advert listed by a specified hiringOrganization. It extends the JobPosting⁸ concept in schema.org and defines essential attributes – including the so:-jobLocation, so:datePosted, and so:hiringOrganization, in addition to the saro:describes to state the saro:jobRole of the saro:JobPosting.

saro:Skill. – a saro:JobPosting links to a set of inferred and explicitly specified saro:Skills, using the relation saro:-requiresSkill. Another key relation is saro:frequencyOfMention, used to specify the number of mentions (occurrence) of a saro:Skill in a saro:JobPosting. saro:Skill extends esco:Concept, which categorizes skills (or competencies) as *job-specific* or *transversal* (cross-sector). SARO further extends this into:

- (1) **saro:JobSpecificSkill:** representing technical skills related to a particular job role, further sub-classed into:
 - (a) **Product:** competence using a particular product (e.g., “Hadoop”).
 - (b) **Topic:** capability in a domain- and/or role-specific topic required to achieve an observable result (e.g., “Data Analytics”).
 - (c) **Tool:** competence in the use of a tool specifically for carrying out technical tasks, e.g., a specific programming language (e.g., “Java”, “Python”) or database type (e.g., “NoSQL”).
- (2) **saro:TransversalSkill:** sector and occupation-independent skills foundational to personal development, often referred to as *soft* skills (e.g., “team-working”).

Picture 1: Saro Explanation as per [2]

Also, helpful and especially for the pre-processing and data cleaning was the paper of I. Karakatsanis et al. [1] which apart from helping with the pre-processing it also Analise the TF-IDF process that I used in my implementation on the Spark.

2.2 Terms Explained

Below I will provide descriptions of some of the terms that were used in the thesis:

Dictionary-Based Entity Extraction

Dictionary-based entity extraction, also known as gazetteer-based matching, serves as a deterministic approach to Named Entity Recognition (NER) by identifying occurrences of predefined terms within a document through direct string matching. At its core, this methodology utilizes a structured lexicon, ranging from simple flat lists to complex hierarchical taxonomies, to scan input text for exact or matches. While the technique is constrained by its inability to generalize to terms that are out of the vocabulary and its sensitivity to linguistic ambiguity, it offers precision and computational efficiency in domains with standardized terms and key words, such as the medical, legal, or technical sectors. Consequently, dictionary-based methods are frequently employed either as a robust baseline for information extraction tasks or as a high-precision component within a hybrid NLP pipeline, bridging the gap between manual curation and statistical inference.

OBIE (Ontology-based Information Extraction)

Ontology-based Information Extraction (OBIE) is a subfield of information extraction that uses ontologies—formal representations of knowledge within a specific domain—to guide the process of identifying and pulling structured data from unstructured or semi-structured text.

While traditional Information Extraction (IE) might just look for names or dates, OBIE identifying the semantic relationships between those entities based on a predefined blueprint of the world.

ISCO (International Standard Classification of Occupations)

Developed and maintained by the International Labour Organization (ILO), the International Standard Classification of Occupations (ISCO) serves as the premier global framework for organizing jobs into a clearly defined set of groups according to the tasks and duties undertaken in the job.

ESCO (European Skills, Competences, Qualifications and Occupations)

ESCO categorizes and defines the specific skills, qualifications, and professional roles essential to the EU's labor market. It functions by mapping the intricate connections between various vocations and the competencies they require, bridging the gap between the educational sector and the workforce [5].

O*NET (Occupational Information Network)

The Occupational Information Network (O*NET) operates as an open-access repository of comprehensive job profiles. It is designed to assist diverse stakeholders—ranging from students and corporate entities to workforce developers—in navigating the complexities of the contemporary United States labor landscape [6].

2.3 Related Work

In the paper “**An Open and Data-driven Taxonomy of Skills Extracted from Online Job Adverts**” [7] develops a data-driven taxonomy of skills mentioned by employers in online job adverts and does not rely on existing models and ontologies such as the O*NET and ESCO. Unlike traditional frameworks that rely on expert opinions, this model utilizes machine learning and network analysis to automatically group skills based on how they appear together in job listings. By treating skills as interconnected points in a graph, the authors can identify specialized clusters and distinguish them from broad, transversal competencies like communication. This algorithmic approach allows the taxonomy to be updated in real-time, reflecting shifting labor market demands and salary trends more accurately than static models. Ultimately, the project aims to provide an open-source tool for policymakers and educators to better understand evolving professional requirements and support worker transitions.

The research paper [8] “**Discovering Job Market Trends with Text Analytics**” details a computational framework for identifying employment trends within the technology sector by using text analytics. The authors utilized automated web scraping to gather data from platforms like Glassdoor and Indeed, ensuring accuracy by cross-referencing company profiles with specific job listings. To process this information, they employed Elasticsearch for keyword indexing and text mining, alongside R for sophisticated data visualization. Their methodology transforms raw text into actionable insights, such as word clouds and time-series graphs, which highlight high-demand skills and certifications. This research paper was a great inspiration, especially for the utilization of Elasticsearch, and

for analyzing the indexing process which essentially is the data-cleaning stage prior for the data to be inserted into elasticsearch.

The research paper **“Extracting knowledge from online sources for software engineering labor market”** [9] a Systematic Mapping Study that explores how digital platforms can be utilized to analyse the software engineering labour market. By reviewing 86 primary studies, the authors identify key online sources like LinkedIn, GitHub, and Stack Overflow that provide real-time data on hard and soft skills. The study categorizes these sources and the data analysis methods used to process them, such as machine learning and natural language processing. It highlights the shifting focus from traditional surveys to digital traces, which offer faster insights into industry trends. Ultimately, the findings serve various stakeholders, including recruiters seeking ideal candidates, universities updating curricula, and policymakers monitoring economic shifts. This comprehensive overview maps the connection between digital knowledge extraction and the evolving demands of the global technology workforce.

In the paper **“A labor market intelligence system for classifying web job vacancies”** the researchers developed WoLMIS (Work Labor Market Intelligence System), a sophisticated labor market intelligence system designed to automatically categorize web job vacancies into a standardized international framework. By utilizing machine learning algorithms like Support Vector Machines and Neural Networks, the platform converts unstructured online postings into actionable data for policymakers and employment analysts. This technology allows for a macro-perspective evaluation of economic trends and labour demands across different European nations, effectively overcoming traditional linguistic and geographic barriers. The authors demonstrate the system’s utility by processing millions of job offers, providing real-time insights that complement slower, traditional statistical surveys. Ultimately, this tool facilitates a deeper understanding of vocational training needs and shifting professional requirements within the modern digital economy. This paper was inspiring about the method used for overcoming the language problem (page 3 - that the issue of languages is can be tackled by implementing a standardized taxonomy). The WoLMIS addresses the inherent challenge of multilingualism in web-based job vacancies by leveraging standardized international taxonomies to facilitate cross-border labor market analysis. Central to this approach is the mapping of heterogeneous job advertisements to the International Standard Classification of Occupations (ISCO), which serves as a lingua franca that renders vacancy datasets comparable

regardless of their original language. To support this, the system incorporates the ESCO multilingual taxonomy, which provides translations of occupation concepts into more than 20 European languages. Furthermore, the WoLMIS architecture is designed for modular extensibility, utilizing dedicated language-specific processing pipelines that can be adapted by selecting appropriate stop-word sets, stemming algorithms, and localized training data. By decoupling the input language from the standardized output classification, WoLMIS overcomes the linguistic limitations of traditional proprietary systems, enabling a comprehensive macro-level perspective on international labour market dynamics.

The research paper **“Research on Information Collection Method of Shipping Job Hunting Based on Web Crawler”** investigates efficient automated techniques for gathering shipping industry recruitment data from the internet using Python-based web crawlers. The authors compare two primary methods: utilizing standard Python libraries and implementing the Scrapy crawler framework to extract information from specialized maritime websites. Their findings reveal that while both approaches successfully collected over 127,000 records, the Scrapy framework was significantly faster, reducing the processing time from 75 minutes to just 45 minutes. The study details the technical architecture of these tools, including components like the Scrapy Engine, Scheduler, and Item Pipeline, which transform unstructured web content into organized data. Ultimately, this work demonstrates how advanced scraping technology can effectively build comprehensive databases for subsequent maritime labor market analysis and data mining. This research was helpful for the designing of the web scraping mechanism and contributed to conclude into Scrapy framework.

The research paper **“Text Mining on Job Advertisement Data: Systematic Process for Detecting Artificial Intelligence Related Jobs”** [10] investigates how text mining online job advertisements can reveal shifts in the demand for artificial intelligence skills within the Finnish labor market. By analyzing nearly half a million job postings from 2013 to 2020, the authors developed a three-tiered classification system to categorize the depth of AI expertise required by employers. The study utilized natural language processing to identify trends across various industrial sectors, linking the findings to firm-level data for a more detailed economic perspective. Results indicate a growing transition from general infrastructure needs to highly specialized AI applications, particularly within the financial

and public sectors. Ultimately, the source argues that job ads serve as a vital real-time metric for tracking how new technologies are adopted across an entire economy.

The research paper “**Data mining approach to monitoring the requirements of the job market A case study**” introduces a data mining framework designed to track global labor market trends by bridging the gap between static databases and real-time job postings. The authors developed a Latent Semantic Indexing (LSI) model to match online advertisements with the O*NET database, allowing for a precise quantification of occupational demand across different industries and regions. By analyzing sectors like oil and gas and banking in the US, UK, and GCC countries, the study identifies specific skill clusters and shifting professional requirements. The findings suggest that this automated approach offers more timely and detailed insights for policymakers and recruiters than traditional government statistics. Ultimately, the method provides a scalable way to monitor how technological advancements and economic shifts redefine the modern workforce.

The research paper “**Evaluation of the trends in jobs and skill sets using data analytics**” [11] examines how automation and artificial intelligence are reshaping the oil and gas job market by shifting demand toward specific skill sets. Using advanced data analytics like Latent Semantic Indexing and Factor Analysis, the authors identify a transition away from manual and routine tasks in favor of interpersonal, technical, and problem-solving abilities. The study highlights a significant skills gap between the specialized requirements of the industry and the general knowledge provided by the educational system. Furthermore, it contrasts regional trends, noting that the GCC market emphasizes engineering roles while the United States prioritizes managerial and human resource positions. Ultimately, the authors propose a digital framework to help policymakers and educators align workforce training with the evolving needs of a technology-driven economy.

The research paper “**Job Analysis System Based on Spark Platform**” [12] introduces a Job Analysis System designed to bridge the gap between vocational education and the rapidly evolving demands of the IT industry. The authors utilize a distributed crawler framework powered by Redis and Zookeeper to harvest real-time employment data from major recruitment websites. This raw information is then processed via the Spark platform to identify essential technical skills and industry trends through big data cleaning and clustering. To make these findings accessible, the system employs microservice

architecture and charts for data visualization, allowing educators to see clear correlations between job requirements and salaries.

The academic study “**Text Mining of Industry 4.0 Job Advertisements**” [13] explores the evolving labor market within Industry 4.0 by utilizing text mining to analyse over 1,400 online job advertisements. The researchers identified a critical shortage of workers skilled in smart factory technologies and sought to map the specific competencies and job profiles currently in demand. Their analysis uncovered eight distinct multidisciplinary roles, ranging from supply chain analysts to systems engineers, highlighting a need for expertise in the Internet of Things (IoT) and big data. The findings suggest that modern industrial roles increasingly require a blend of electrical engineering, software development, and traditional management. This research was inspiring for developing TF-IDF algorithm that can detect the unique features of specific roles. Specifically, following data cleaning, researchers employ frequency analysis and metrics such as TF-IDF (Term Frequency-Inverse Document Frequency) to evaluate the relative importance of specific phrases within a corpus. To move beyond simple word counts, advanced techniques like hierarchical cluster analysis are often used to identify latent associations between phrases. By applying similarity measures such as Jaccard’s coefficient, these algorithms group related terms into overarching "topics" or profiles, providing a visual and statistical landscape of the subject matter. Ultimately, this data-driven approach is highly effective for weak-signal detection, allowing researchers to identify emerging trends and subtle shifts in a field—such as rapidly evolving job competencies

The research paper “**Streaming Twitter Data Analysis Using Spark For Effective Job Search**” [14] details the development of a scalable system designed to identify and organize job advertisements from the massive stream of daily Twitter data. By utilizing Apache Spark, the authors provide a high-performance alternative to traditional Hadoop frameworks, enabling near real-time processing of millions of tweets. The proposed model uses machine learning to filter out irrelevant posts and categorize legitimate vacancies into sectors such as IT, finance, and healthcare. Results indicate that a significant volume of career opportunities is shared via social media, making automated classification essential for efficient job searches. Ultimately, the study demonstrates how Spark Streaming and Spark SQL can be deployed in cloud environments to help users find employment without needing to follow specific corporate accounts.

The research **“From Data to Insight Transforming Online Job Postings into Labor-Market Intelligence”** [15] presents a systematic methodology for converting vast collections of online job postings into actionable labor-market intelligence. By utilizing Natural Language Processing (NLP) and machine learning instead of large language models, the authors address core challenges such as data deduplication, the normalization of job titles, and the extraction of industry-specific metadata. The framework is validated through two distinct use cases, including an analysis of the Greek tourism industry during the pandemic and the development of a data-driven skills taxonomy. The findings demonstrate that these automated techniques can effectively identify real-time shifts in employment types, education requirements, and emerging skill demands. Ultimately, the study provides a robust blueprint for policymakers and businesses to monitor economic trends and workforce development needs with high precision and transparency.

The research article **“Research using job advertisements: A methodological assessment”** [16] presents a methodological assessment of how job advertisements are utilized as data within the field of library and information science. The authors conduct a systematic review of over one hundred peer-reviewed studies to identify patterns in how researchers select, collect, and analyse these postings. While job ads are valuable for tracking workforce trends and informing curriculum development, the study highlights a lack of consistency and rigor in current research practices. Findings indicate a shift toward online data sources and more sophisticated analytical techniques, though issues regarding data representativeness remain. Ultimately, the source serves as a guide for improving the credibility and validity of future labour market investigations.

The survey **“Text Classification Algorithms: A Survey”** provides a comprehensive analysis of the text classification pipeline, detailing the evolution from raw data to actionable insights. The authors outline four primary stages: text preprocessing, feature extraction, dimensionality reduction, and model evaluation. The text explores various methodologies, ranging from traditional weighted word techniques like TF-IDF to advanced deep learning architectures that capture complex data relationships. Additionally, it addresses practical challenges such as noise removal and the limitations of various algorithms in real-world applications like spam filtering and recommender systems. By examining both statistical methods and modern neural networks, the source serves as a technical guide for optimizing natural language processing workflows. This survey was particularly helpful in the text pre-processing which involves data cleaning, and for the TF-IDF part.

A site that was used extensively for the Exploratory Data Analysis (EDA) was Kaggle [17]. Established in 2010 and acquired by Google in 2017 Kaggle has evolved into the preeminent platform for competitive data science. It serves as a critical repository for high-fidelity, real-world datasets and a "crowdsourced" validation ground for state-of-the-art (SOTA) algorithms. The platform's competition format facilitates a unique form of peer-reviewed empirical testing, where diverse approaches—ranging from traditional statistical models to complex ensemble techniques like XGBoost and Deep Learning architectures—are evaluated against objective, hidden test sets. This creates a transparent leaderboard that often defines the empirical upper bounds of predictive performance for specific domains, such as bioinformatics, financial forecasting, and computer vision. Furthermore, the platform's Kernels (now Kaggle Notebooks) provide a reproducible environment that fosters "open science" by allowing researchers to inspect, fork, and build upon the code and exploratory data analysis (EDA) of a global community.

Kaggle was used in this thesis especially at the TF-IDF analysis, the main algorithm was taken from the kaggle notebook: "NLP GloVe, BERT, TF-IDF, LSTM Explained" [18] it was also quite helpful for the pre-processing stage.

Stackoverflow Survey: A very important source for the hard skill, technologies and products statistics of the IT sector on a global level is the annual report of stack overflow: <https://survey.stackoverflow.co/2025/technology> [19] where a survey is taking place every year, the most recent is at 2025 and has 6 parts the 'Developers' part which analyses "What we know about the global community", the technology part which will be analysed more thoroughly below, the AI part which "seeks the insight into the real sentiments behind the surge in AI popularity, whether it is making a real impact in the way developers work or is it all hype", the work part where "Developers tell us about workplace trends and offer some transparency about salary for different roles and experience levels", the Stack Overflow part where there are questions related to the platform usage and how connected the people using it feel and lastly is the part where there is an explanation on the methodology and the participants of the survey.

The survey overall depicts the trends in the field, however the part which will be used extensively for this thesis is the technology section, which comprises of the following three main categories of questions:

1. What are the most popular technologies
2. What other technologies or products are interested and desire to use in the future.

3. Technologies worked with vs want to work with

In the survey of 2025 consists of 49,009 responses with participants from 177 countries. Consequently, this survey is an important metric for not only the tech stack that is used but also for the hard skills that are in demand derived from what employees and companies are utilising in production.

3 Extraction - Web Scraping

In this chapter I will analyze the web scraping part of the application, how I developed the scrapy 'spiders' for each site, what data cleaning, formatting and normalization I implemented.

3.1 Web Crawling

The process of automatically browsing the internet to discover and collect information from web pages through software programs is called web crawling. The web crawling programs that was developed in this thesis have a similar structure, which is: first, the crawling of a "Search Page" whereas search means the result of a search query on a web page which essentially gives links of the relative items that we are scraping. Second, the processing of all the html pages that the "Search Page" gave, and the storage of only the desired information that we defined in the program.

Web scraping is increasingly recognized as a valid methodology for public data collection, provided the data is not protected by technical 'gates' (logins/encryption) and the collection process does not impair the functionality of the host infrastructure. However, the 'public' nature of the data does not grant an absolute right to its extraction; researchers must balance the transparency of the web with the proprietary interests of the data controllers and the privacy rights of the subjects involved.

In this thesis crawling was conducted by following the general guidelines of the frequency of the requests and the data that are extracted are publicly available data of which their access does not require an account or trespasses any 'gate' except from 'robots.txt' in some cases like LinkedIn.com.

3.1.1 Web Crawler

A Web Crawler (also known as a web robot) is a system designed for the bulk downloading of web pages to create a searchable index of the internet (Olston & Najork, 2010). The crawling process begins with a "frontier" of seed URLs; the crawler downloads these pages, identifies new hyperlinks, and adds them to the frontier to be visited in a recursive cycle (Bahrami et al., 2015).

3.1.2 Web Scraper

A Web Scraper is a software program that mimics human browsing behavior to extract large amounts of specific data that would otherwise be difficult to collect manually (Solove & Hartzog, 2024). While a crawler finds the page, the scraper parses the HTML (or other unstructured formats) to extract targeted data points—such as prices, contact details, or profile information—and saves them into structured databases like JSON or CSV (Sánchez Paipilla et al., 2022; Tanasa et al., 2024).

3.1.3 Web Spider

A Web Spider is often used as a synonym for a web crawler (Bahrami et al., 2015). However, some technical contexts define the "spider" as the specific agent or software program that performs the traversal, whereas "crawling" refers to the overarching algorithmic process (Sánchez Paipilla et al., 2022). The term "spidering" specifically denotes the methodical, automated way the program "surfs" from one link to another to download site content to a local disk (Young & Dean, 2012).

3.2 Scrapy

Scrapy framework was used as the main data extraction method to make http requests to the 4 sites (kariera.gr, linkedin.com, skywalker.gr, jobfind.gr) and then store the valuable attributes from the pages that was defined, which are common for all 4 of the sites. “[3] Scrapy is a fast high-level web crawling and web scraping framework, used to crawl websites and extract structured data from their pages. It can be used for a wide range of purposes, from data mining to monitoring and automated testing “.

In the scrapy framework, one of the main tools that you can use is 'spiders' which are customized algorithms that extract specific information from HTML pages. "Spiders are classes which define how a certain site (or a group of sites) will be scraped, including

how to perform the crawl (i.e. follow links) and how to extract structured data from their pages (i.e. scraping items). In other words, Spiders are the place where you define the custom behavior for crawling and parsing pages for a particular site (or, in some cases, a group of sites).”

Each site has its own dedicated custom ‘Spider’ and each ‘Spider’ has some definitive attributes which are prerequisites to successfully crawl pages. So, each ‘Spider’ has one list of ‘allowed domains’ which have one string element that is the domain of each site, e.g. LinkedIn.com, then each ‘Spider’ has a ‘start URLs’ list that has the URLs that the spider will begin to crawl, and lastly there are the ‘rules’ which are a set of rules that the spider will follow, for example the most common rule is the ‘allow=’X’...’ which will program the spider to crawl every html element that has the string “X” in it, in the particular spider of ‘kariera.gr’ it was used like this: ‘allow=r’/jobs/’,’ which means: From the page in ‘start_url’ crawl every html ‘href’ element that the link has the string ‘/job/’ in it. In this use case the ‘Spider’ will crawl all the job posts that are on the search page.

3.3 General Overview of the Crawling Methods

In this chapter the technical aspects of how web crawling was made possible, and the configurations will be discussed.

3.3.1 Configuration and Initialization of Scrapy

First, I created a Python virtual environment² in which I downloaded all the dependencies and libraries necessary including Scrapy. Scrapy can be installed with pip³ in the Python virtual environment, when the installation finishes, we can check if the scrapy is indeed installed by using this command: ‘*scrapy version*’ this will print the current version of

² The venv module supports creating lightweight “virtual environments”, each with their own independent set of Python packages installed in their site directories. A virtual environment is created on top of an existing Python installation, known as the virtual environment’s “base” Python, and by default is isolated from the packages in the base environment, so that only those explicitly installed in the virtual environment are available. [31]

³ pip (also known by Python 3’s alias pip3) is a package manager (package management system) written in Python and is used to install and manage software packages.[4] The Python Software Foundation recommends using pip to install Python applications and its dependencies during deployment.[5] Pip connects to an online software repository of public packages, named the Python Package Index (PyPI). [32]

scrapy if it is installed correctly. After installing Scrapy, a project folder with all the necessary files must be created. This is a standardized directory that scrapy requires managing settings, items and pipelines. The creation of that project folder can be achieved with this command: *scrapy startproject myprojectname*, more specifically this command creates a root directory containing 'scrapy.cfg' file and a Python package folder with the spider logic and middleware configurations.

After the creation of the project structure, we can then proceed to create the 'Spider' objects which as mentioned above are the classes that contain the logic for crawling the links and parsing the HTML pages.

3.3.2 Scrapy Settings

Some settings were necessary for the scraping process to be successful; those are the default request headers, the Concurrency and throttling settings, analyses on why those are necessary follows:

The 'default request headers' is used to include every request for a specific header with a 'User-Agent'⁴ set. Headers in those requests are important because the request appears like they are being conducted by a browser and not a bot⁵. If this is not defined in the settings, the requests have the 'User Agent' set by Scrapy, which is "Scrapy/VERSION (+https://scrapy.org)" making it obvious that the request is by Scrapy, resulting in some cases the crawling not being possible because user agents like the last mentioned are flagged. An example of the header is below:

```
DEFAULT_REQUEST_HEADERS = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 Chrome/129.0 Safari/537.36', 'Accept-Language': 'en-US,en;q=0.9',}
```

This is pretending to be a normal Chrome browser on 64-bit Windows 10.

The concurrency settings are 4 settings that are configuring the frequency of the http requests, more specifically: The maximum concurrent requests, the time delay between two requests, and the 'RANDOMIZE_DOWNLOAD_DELAY' which as its name indicates

⁴ The User-Agent header is an HTTP header intended to identify the user agent responsible for making a given HTTP request. [33]

⁵With bot meaning a crawler or a script

randomizes the delay time by multiplying the download delay from 0.5 to 1.5. Below we can see those settings:

```
CONCURRENT_REQUESTS = 2
```

```
DOWNLOAD_DELAY = 10
```

```
RANDOMIZE_DOWNLOAD_DELAY = True
```

Concluding with the settings, the `autothrottle_enabled` setting was set to `True`, this was done to reduce the chance of getting flagged as a bot, automatically adjust Scrapy to the optimum crawling speed, so the user doesn't have to tune the download delays to find the optimum one. [20]

The last setting which was necessary at least on LinkedIn was to ignore `robots.txt`, as mentioned above when considering `robots.txt` it was not possible to scrape LinkedIn, so it was a necessary measure to successfully extract the data for that thesis, above we can see also the setting from the `settings.py`:

```
ROBOTSTXT_OBEY = False
```

3.3.3 Network Anonymization and Anti-Blocking Measures

To ensure the continuity and reliability of the data extraction process, a Virtual Private Network (VPN) was used. Web scraping without network masking renders the host machine's static IP address identifiable to the target server's.

Frequent HTTP requests originating from a single source can trigger rate-limiting mechanisms or permanent IP bans. By rotating the IP address via a VPN for each distinct crawling session, the system simulated requests from disparate geographic locations. This strategy mitigated the risk of detection during long-duration extraction tasks.

3.3.4 Testing phase for locating the data within the DOM⁶

The initial phase of the parsing logic involved the systematic identification of target data points within the Document Object Model (DOM). As the DOM represents an HTML

⁶ **DOM:** The Document Object Model (DOM) is a cross-platform and language-independent API that treats an HTML or XML document as a tree structure wherein each node is an object representing a part of the document. The DOM represents a document with a logical tree. Each branch of the tree ends in a node, and each node contains objects. DOM

document as a hierarchical tree structure, programmatic access to specific nodes is essential for accurate data extraction.

The CCS Selector: Used for efficient targeting based on element attributes, classes, and IDs, providing a high degree of readability and speed.

XPath (XML Path Language): A more powerful language that can navigate bidirectionally, hence finding a parent element based on its child, which with CSS is not possible. Regardless of which of the two is used, typically you can target elements in the HTML using the below four attributes.

Parse by HTML Tag: Parses elements based on their HTML tag category, this works when you need all instances of a specific type like all HTML list items (li).

Parse By Class Name: Parses elements based on the class attribute, suitable for scraping items which have a distinct visual format (e.g. search result list)

Parse By Element ID: Targets a specific, unique element. Based on HTML standards an ID should only be used once per page.

Parse By Attribute: Targets elements based on other attributes and not Class or ID, for example src, href, title, or custom data attributes.

A fundamental component of Scrapy architecture is the *scrapy shell* `<url>`. The Shell is the interactive tool developers use to quickly test CSS and XPath selectors before writing them into the parse method of their spider. Scrapy shell is important because you can test code in a terminal like way on a small-scale crawling one page at a time, prior to building the spider script that scrapes at a scale.

One of the most useful methods when using the shell is the ‘*Parse*’⁷ method. The ‘*Parse*’ serves as the default callback function for processing downloaded web pages. Within this method, CSS Selectors or XPath expressions can be utilized to pinpoint and extract specific data elements. This is made by executing an HTTP request to one ‘*item page*’⁸ and

methods allow programmatic access to the tree; with them one can change the structure, style or content of a document. [44]

⁷ Parse Method: Serves as the default callback function for processing downloaded web pages.

⁸ Item Page: In this implementation is a job post, all item pages are identical and are accessible via the search results page

then applying the selectors to the *'response'*⁹. It is necessary for every site that we scrape to have concluded not only on which elements are those that we aim for, but also to decide how to scrape them, because one html text can be identified by many ways and some of them could cause issues when scraping in scale. Generally, we can say that, for scraping the most efficient ways are those that directly identify the item (e.g., via unique IDs or class names) and not those that lead to the item relatively¹⁰ which are highly susceptible to breaking when website layouts change.

During the development phase, this extraction logic can be rapidly tested and refined using the Scrapy Shell, significantly accelerating the HTML tag identification process.

3.4 The Web Crawlers

In this chapter, the 4 web crawlers will be analyzed. Before that it is important to review the sites that was choosed, what is the amount of the contribution of each to the dataset and what is the quality of the data each site gave.

After reviewing several job portal sites in Greece I concluded that Kariera.gr, Skywalker.gr and Jobfind.gr are representative examples for the job portal sites in Greece. The criteria were mainly based on the quality and the quantity of the job posts that are hosting relative to the scope of the thesis, which is the IT sector, other criteria were also how many total job posts they get and the versatility of the sectors in the sense that since many employers are using it, there must be correspondent usability. However, kariera.gr stands out as the most usable of the employees of the IT sector since it not only has the most job posts in the sector, as we will see in the table below, but it is also updated frequently.

Table 1: Job Posts in Greek Sites at 4/10/2026

	Total Posts (In Greece)	IT Sector
Skywalker.gr	3027	79
Jobfind.gr	11029	144
Kariera.gr	6998	234

⁹ Response Object: The response is an object instantiated by Scrapy that contains the downloaded HTML content and metadata of the requested page, which is then parsed for data extraction.

¹⁰ Relative Selectors: Extracting an HTML element based solely on its hierarchical or spatial relationship to other surrounding elements in the Document Object Model (DOM).

As mentioned, these sites have a noticeable usability which must be taken into consideration when we investigate a sector in Greece. However, our research would not be objective if we only rely on local sites, since the most important job portals are those with a worldwide user base, and among them the most usable with the most engaged users is LinkedIn. Of course, there are other sites that are used extensively for job posts in Greece, and even more than the three Greek job portals mentioned, however the two most in use after LinkedIn which are indeed.com and Glassdoor.com had a strict policy¹¹ for accessing them in public¹², therefore scraping them was not feasible.

Kariera.gr

The site “kariera.gr” was the first crawler that I developed, after crawling the search page with a specific string search specified, for example searching ‘Software Engineer’, the results were usually relative but there was two disadvantages, first was that the quantity of the results, usually were not fully filling a page with the maximum amount of job posts per page which is 100 per search result making this method lacking in quantitative efficiency, and the second is that the dataset were filling with specified job posts instead of a sector representative portion making the end results biased. That is why I decided to crawl ‘by sector’, hence instead of using a search result, I used specific category of results which represents the industry of job posts, those were the jobs from the IT industry, below is the link:

kariera.gr/en/jobs?industry=SOFTWARE_VENDOR_INFORMATION_TECHNOLOGY&page=0&limit=100

Skywalker.gr

‘Skywalker.gr’ is a simpler and smaller site compared with ‘kariera.gr’. It is possible also in ‘Skywalker.gr’ to crawl by search results, however I followed the method I mentioned above which is the crawling of a specific category which is the IT sector. In ‘Skywalker.gr’ there are two basic categories, the group by location and the group by sector, the category that was crawled from that site is ‘Πληροφορική’. Below is the link:

[skywalker.gr/el/aggelies-ergasias?category=01hdtm1ff0g1rdef228eftwthm"\]](http://skywalker.gr/el/aggelies-ergasias?category=01hdtm1ff0g1rdef228eftwthm)

¹¹ Glassdoor has captchas and Cloudflare

¹² With public access meaning access without an account.

Jobfind.gr

Jobfind.gr is also a simple site like Skywalker.gr, again the same method as the previous sites. Also, on this site I crawl by sector on a specific category of the site, the link of which is the below:

jobfind.gr/JobAds/Pliroforiki_Tilepikoinonies/all/GR/Theseis_Ergasias'

LinkedIn.com

LinkedIn Is a more complex site compared with the other three, and this is expected since it is the most popular job portal site used worldwide, “As of 2026, LinkedIn has more than 1.2 billion registered members from over 200 countries and territories.” [21] .

Scraping LinkedIn in this thesis went through two methods. The first method was like the other sites, hence crawling a 'search page' then all the relative links and then all the relative content, not by crawling a 'category page' but rather by crawling with 'search results' for example searching 'Technical Support Engineer' and crawling the job posts occurred from the results, below there is an example of the links crawled:

linkedin.com/jobs/search?keywords=application%20support&location=Greece&geoId=104677530&trk=public_jobs_jobs-search-bar_search-submit&position=1&pageNum=0

3.4.1 LinkedIn Crawling Issue

The method mentioned above was followed for about a month up until it was not possible, at some point LinkedIn decreased the number of job posts returned to the search page (Without being logged in with an account) result to only 10 job posts maximum, where before this change the results were approximately 60. There was not a change in the code or the method of crawling used while experiencing this change, so possibly LinkedIn made that change to avoid web crawlers and at the same time decrease the load to their servers for requests made without an account¹³. Obviously, this hindered the process, making it impossible to crawl the necessary amount of job posts to have valuable results.

A solution for this was the second method that I developed, which essentially is the utilization of the fact that LinkedIn does not limit the unique job post pages of requests coming from one IP for specific job posts. Despite the limit in the 'search page' results, the

¹³ The requests in all sites were made without an account, using the 'public space' of the sites.

usage of job posts is not limited, at least with the crawling settings mentioned above¹⁴. The reason for that is possibly because LinkedIn wants unique job posts to be accessible to as many people as possible to expand its popularity, or it is not by design and it is due to the fact that scraping unique posts have not arisen any alerts or issues. So, to clarify, there is no limitation for making requests to specific job posts, as long as the respective job posts ids are available.

After discovering this, I proceeded to the next step which was how to get LinkedIn lists of links (actually lists of ids) to crawl. My first approach was to log in LinkedIn with my account, search for the job category I want to scrape, then store the page locally in HTML and load the page with the spider set to crawl the first page locally and then crawl its links, essentially this was same approach with the other job portal crawlers but instead of making the first http request to the internet, it is doing it to a locally stored HTML file. This method worked and gave results; however, it is a cumbersome method that consumes a lot of time and preparation.

That's why I developed the final method which was using the links that are indicated in each job post. More specifically, each job post in LinkedIn recommends job posts relative to the current one, each job post can have up to 37 maximum recommended job posts, so I developed the crawler that:

1. Reads a local text with the LinkedIn job posts ids and stores those ids in a list.
2. Iterates through that list and scrapes all the job posts ids.
3. Scrapes also the ids of each job post crawled and storing them in custom key-value pairs inside the JSON file made for LinkedIn, each job post has its list of job ids.

With this method the result is a JSON file that for each job post there is a list of job post ids ranging from 1-37. After applying this method for some crawls, it became obvious that the number of ids extracted is way above the crawling capabilities of this project, for example, the available unique links for scraping are more than the database job posts (3184), counting above 4.500 ids and the job.

¹⁴ In the same chapter (3.3) where the settings are analyzed

4 Data Transformation

In this chapter we will discuss the ‘transform’ part of the overall process, transform stands for the pre-processing stage of the data analytics which can have the transformations of ‘Data Cleaning’, ‘type correction’ etc. This section can be defined at the stage right after the scraping process where the data are outputted in JSON format until the stage where the data are inserted into the database.

4.1 Transformations Within the Crawlers

Some Transformations were added within the crawlers to make the overall process more efficient, since some transformations are fixed it was more robust to embed them into the crawling process. These processes were the following:

4.1.1 Job ID Extraction and Cleaning

In the database I use 2 kinds of IDs, one is the Primary Key and it is used mostly for troubleshooting and identifying a job post between different environments, e.g. elasticsearch and postgresQL. The other ID is the job ID, and this helps indicate the job posts between the database and the job portal, making it very useful for the avoidance of entering duplicate job posts and for reference with the job posts on the site, both of those fields will be analyzed in chapter 5.

The cleaning of the id for each site, usually the id was embedded inside the link, which needed processing to extract the id, so the below one-line code was written to remove the all the link information except from the id.

```
job_id = response.url.split('/')[0].split('?')[0]
```

This paradigm of the extraction was taken from the *kariera.gr* site, as we can see this process has two main parts of processing the url, first the link is splitted based on the dashes keeping the last element (*.split('/')[0]*) and second splitting again wherever there is question mark and keeping the first element (*.split('?')[0]*).

This is the general method of id extraction on each site, for *skywalker.gr* and *jobfind.gr* the process was simpler since it was splitted only one time, in *LinkedIn.com* the URL

structure was more complex and it needed 3 splitting methods, as we can see the id extraction from the first spider¹⁵ of LinkedIn below.

```
"job_id": response.url.split('/')[1].split('?')[0].split('-')[1],
```

4.1.2 Description Field Cleaning

After scraping different sites, it came across as necessary to process the job description field in a way that it is homogenous, hence a field with text and no special characters. This process eventually evolved into a two-section solution the first section being at the ‘Transform’ stage¹⁶ and the second section being at Data Analysis stage. Here we will see the first section of this process, at this stage right after scraping the job description there are html tags especially the new line element ‘\n’ and multiple spaces in a row which must be removed. For this reason, a ‘cleaning’ script was developed, this script has multiple functions that are controlled by an input string. This script has two inputs, one as mentioned is the ‘control’ and the other is the JSON file that will be processed. With the input ‘description’ the cleaning process mentioned is activated, removing trailing spaces and new line elements. This can be seen in the picture below.

```
8 if sys.argv[2] == "description":
9     for job_post in data:
10         job_post['job_description'] = job_post['job_description'].strip().replace('\n', ' ')
```

Picture 2: As we can see when the input is “description” the ‘cleaning’ process is taken

4.1.3 Concatenated Words Scraping Issue

At the beginning of scraping with the first method, there was an issue which hindered the process and affected a part of the dataset. The issue was the concatenation of two words from different sentences in the job description which at the primary source¹⁷ was separated by a new line element ‘\n’ or ‘\r\n’.

This happened because the job description was scraped as a list of sentences then joined the sentences (list elements) and mistakenly replaced the new line element \n with no space ‘’ resulting in concatenating the two sentences.

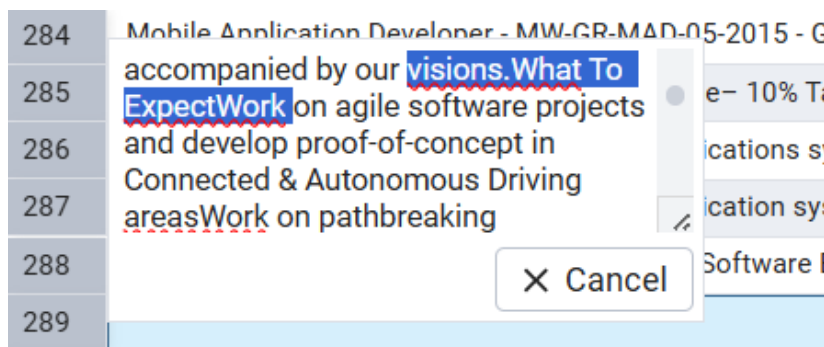
¹⁵ The two methods developed are analyzed in chapter 3.4.1 “LinkedIn Crawling Issue”

¹⁶ The definition of this being at the beginning of the chapter.

¹⁷ Where the primary source is the job description field of the job post.

So, a part of the dataset has concatenated two sentences; most of those sentences have a punctuation mark between them, because in most cases the new line indicates a new sentence and therefore the punctuation mark is necessary. However, there were cases where there was not a punctuation mark or any other character indicating a new line.

The cases where there was a punctuation mark between two words was not an issue, since all the punctuations are replaced with space character ' ' for any data analysis conducted, but the cases where there was no punctuation or any other distinct character separating one word from the other was problematic. An example of those two cases can be seen in the picture below, which is a print-screen taken from a job description field of a job post in the database:



Picture 3: Here we see the case where there is a punctuation which is the 'visions.What' and the problematic case where the words are concatenated 'ExpectedWork'

As I mentioned above, this is causing an issue with any data analysis method since the concatenated words resulting in 'canceling' the words that are concatenated and making a new word that adds noise to the results. The most crucial effect to the analysis is the cancellation of the two words, essentially this greatly decreases the quality of the results because some of the words concatenated are not just noise or stop words but are part of the objects that we are investigating. The cause of this issue was the scraping process, and specifically where the job posts are parsed as we can see below:

```
41 def parse_job(self, response):
42     # Scrape the jobs inside the links
43     descriptionList = response.css('.show-more-less-html *::text').getall()
```

Picture 4: Implementation of Text Extraction Logic > The parse_job method demonstrates the extraction of job descriptions using the scrapy response object. A CSS selector is applied to target the .show-more-less-html class, which encapsulates the primary job content. To ensure comprehensive data collection, the *::text pseudo-element is utilized to recursively extract text from all nested child nodes. Finally, the .getall() method serializes these extracted fragments into a structured Python list, stored in the descriptionList variable.


```

75  # the following line was like below, and it was causing issues with words, 2 separate lines where concat
76  # ''.join(descriptionList).strip().replace('\n', ' ').replace('Show more', ' ').replace('Show less', ' ').replace(' ', ' '),

```

Picture 5: This above in the 76th line was causing the issue which is now commented:

The above line was replaced with the below line (56 & 57) where the ‘.join was replaced by ‘.join:

```

59  description_cleaned= ''.join(description_list).strip().replace('\n', ' ').replace('Show more', ' ').replace('Show less', ' ')

```

Picture 6: Here is the correction of the concatenated words issue.

This issue has affected a portion of the dataset so apart from the above solution which resolves the issue, another solution was necessary for the data with concatenated words that are already stored. This solution was an algorithm which was implemented in all the data analysis pipelines, both in the Virtual Machine and in the Jupyter Notebook. Below is this algorithm, this cell was almost identical in all implementations:

```

!!! This stage needs evaluation !!! later I will make one more column like "job_description_tokens" to validate with before-after logic

# Here the exclusion_list is used to not separate words that are expected to have a capital letter in the
# !!! The Exclusion list does not work yet, however the seaparation of camelCase is done successfully
from pyspark.sql.functions import expr, col, concat_ws

exclusion_list = [
    'JavaScript', 'Javascript', 'TypeScript', 'typescript',
    'postgreSQL', 'PostgreSQL', 'MySQL', 'NoSQL', 'nosql',
    'DataFrames', 'iRepair', 'SoftOne', 'TestNG', 'LLaMa'
]

exclusions_sql = ", ".join(f'({x})' for x in exclusion_list)

# Here are removed the combined english words, the combined greek with english words and the combined greek words
# The critiria for all of the are the lower case to uppercase
split_regex = {
    r'(?<[a-z])(?=[A-Z])',
    r'|(?<[A-Za-z])(?=[\u0370-\u03FF])',
    r'|(?<[\u0370-\u03FF])(?=[A-Za-z])',
    r'|(?<[\u0391-\u03A9])(?=[\u03B1-\u03C9])'
}

df_separated = df_desc_clean.withColumn(
    "job_description_separated",
    expr(f"""
        flatten(
            transform(
                split(job_description, ' '),
                w => CASE
                    WHEN w IN ({exclusions_sql})
                    THEN array(w)
                    ELSE split(w, '{split_regex}')
                END
            )
        )
    """)
)

df_separated = df_separated.drop(col("job_description"))

#print(df_separated.filter(col("id")==1434).take(1))
print(df_separated.printSchema())

```

Picture 7: Separator Algorithm

As we can see separates all the words that they have a capital letter after a lower-case letter, this process also called “CamelCase splitting”. This definition of concatenated words was because the new line element is the beginning of a sentence, so there must be a capital letter in the beginning, as mentioned also a dot was expected which would have made this process much simpler but this was not the case for many job posts.

There are also some words that are excluded from the separation process, there are two main cases of excluded words, those are:

1. Well known technology terms that are camel cased and were expected, such as 'JavaScript', 'PostgreSQL', etc.
2. Words that are camel cased but were not expected, since they were not 'established' terms, hence camel case naming that were not well known, so one has to first find those in the dataset and then include them in the exclusion list after investigating their legitimacy, an example is 'iRepair' which is a name of a business.

The result of this process can be seen in the picture below; we can see that the job_description column has the concatenated words 'DescriptionAre' and the output of the de-concatenation algorithm which is the 'Description Are':

id	site	job_description	clean_words	job_description_tokens	job_description_separated
0	25	kariera.gr	Company DescriptionAre you ready to shape the ...	[Company, Description, Are, you, ready, to, sh...	Company Description Are you ready to shape the...
1	37	kariera.gr	Would you like to be part of a team focused on...	[Would, you, like, to, be, part, of, a, team, ...	Would you like to be part of a team focused on...

Picture 8: Result of the de-concatenation algorithm

4.1.4 The List Extraction Issue

At the testing phase in some sites, I stumbled upon an issue regarding the extraction of some fields. The problem was that for the sites LinkedIn.com, Skywalker.gr and Kari-
era.gr the elements job sector and employment type were accessible via a list element in the html, with Skywalker.gr having also the company title as a list element. This is html element extraction by *Relative Selectors*¹⁸ and not by directly identifying the element by id or class name, the problem that this causes is that those list elements are not always in the same order, if for example an element was empty the order of the list would change, resulting in having the elements inserted in the wrong fields, as we can see below:

workplace text	employment_type text	job_sector text
Athens	Sales / Business Development / Account Management	Partly remote

Picture 9: Result of the list extraction issue, the content of job_sector should have been in the employment_type and vice versa.

¹⁸ Explained on footnote 10

This in LinkedIn was not actually a problem because the elements of that list were well organized for all the job posts and the list queue was always the same regardless of missing values, however for Skywalker.gr and Kariera.gr it proved to be slightly problematic. This issue is resolved simply by swapping the data fields with an update query, the simplicity of the solution also relies in the fact that the *job_sector* is a field with expected values. When we know the site that it was extracted from, we can expect specific values to that field e.g. Full-time, Part-time, Partly remote etc. However, the permanent solution is to identify the specific elements for each site and extract by id or class name where it is possible.

4.1.5 Data Insertion to the Database

At the end of the transformation phase, the processed JSON data is inserted into the PostgreSQL database using a Python script. The script uses *psycopg* [22] adapter, which serves as the interface between the python environment and the PostgreSQL database, handling the underlying network communication and data type conversions.

To ensure robust data insertion into the database, the script utilizes Python context managers for both database connections and cursors. This choice ensures that network resources are safely released upon completion or if the transaction terminates unexpectedly.

The data insertion process involves parameterized queries¹⁹. Rather than concatenating strings directly into the SQL command, data is passed as separate parameters. This approach is crucial for data integrity, as it automatically escapes special characters frequently found in job descriptions²⁰, thereby preventing syntax errors and adhering to secure coding standards against SQL injections.

Finally, the script implements a row-by-row transaction strategy. Each insertion command is individually executed and committed within an isolated *try-except*²¹ block. While executing bulk inserts (e.g. using COPY) would yield faster execution, this granular approach prioritizes pipeline resilience. If a specific job post violates a database constraint,

¹⁹ Parameterized queries: VALUES (%s, %s, ...)

²⁰ Such as quotation marks or apostrophes

²¹ Try-except is used for handling errors, if a specific type of error occurs then a clause of code runs defined for that specific type of error [42]

the adapter catches the specific psycopg error, logs the event, and allows the continuous insertion of the remaining dataset without failing the entire batch process.

Before the data are inserted, it checks two things:

- 1) **Duplicate Job Posts:** Iterates through the list of the extracted job posts and does a duplication check, the job post ids that are already in the database are excluded and are not inserted in the database, this process is analyzed in the next chapter “Data Duplication Check”.
- 2) **Workplace Normalization:** Checks for all the job posts if the workplace field is normalized. This is an important process that defines whether the data are inserted or abort the insertion. If any of the elements in the workplace field is not normalized, the insertion will be aborted all together. This process will be defined in chapter five²².

The following code is first the installment of psycopg library and the code for connecting with the database.

```
pip install "psycopg[binary]"
```

```
with psycopg.connect("dbname=jobapplication user=postgres password=testsudo*") as conn:
conn = psycopg.connect(
    host="localhost",
    port=5432,
    dbname="jobapplication",
    user="postgres",
    password="testsudo*"
)
```

Data Duplication Check

Within the data insertion script there is a part of the code that checks for duplicate job posts by utilizing the *job_id*²³ field. So, every new job post that is to be inserted in the database passes a check of duplication, if a job post with a specific job id exists in the database the same job post will not be inserted again no matter how many times it will be

²² This process is thoroughly analyzed in the chapter 5.3.1 “REGIONAL NORMALISATION PREPROCESS”

²³ The *job_id* field will be further analyzed on the next chapter 5.2 “The Schema”

scrapped. This was a very important part of the process because the same job posts could be scrapped not only simply because it would still be present to same page that is scrapped after a period of time, but also because some job posts were re-posted and most of the times with little to no changes to the job post itself. Some could argue that the re-posting is a legitimate information worth included in this kind of research, which is true, however for this thesis this kind of information is out of the scope, and it focuses only on the unique job posts, therefore any job posts with the same id would not be inserted no matter how many times it would be re-posted.

```
49         for id_value in ids_in_json:
50             # checks if the ids_in_json is in the ids_in_db
51             if id_value not in ids_in_db:
52                 post_to_be_inserted = find_posts_by_id(data, id_value)
53             try:
54                 cur.execute("""
55                     INSERT INTO job_posts_1 (job_id, job_title
```

Picture 10: The check of the duplicate job posts is done at the line 51, in the 'if' statement.

5 Database Architecture

In this chapter, I will analyze the ‘load’ phase of the pipeline which involves the database, the execution scripts responsible for data insertion, and the structural design of the database schema.

5.1 The Database

For the core storage of this project, PostgreSQL was selected as the Relational Database Management System (RDBMS). The primary requirement for the system was a transactional database that enforces the ACID²⁴ properties. Specifically, the system required strong Atomicity to ensure that complex data-loading scripts are treated as single, indivisible operations, and Consistency to prevent any partial or corrupted data from being committed to the schema in the event of a failure.

5.2 The Schema

The foundational design of the database schema was adapted from the framework proposed in [15]. The fields I defined from the early development of the thesis were: Job Title, Job Description, Company/Employers Name, Workplace, Employment Type, and Date. Eventually the core of the database schema are the fields mentioned, together with some fields that were necessary for the functional integration between the database and the data analysis environments including several operational metadata fields, those are: specific identification keys, Job Sector, Site, and URL.

²⁴ ACID: Atomicity, Consistency, Isolation, Durability [35] As ACID we define databases that are designed to focus inserting data with rules defined in the creation of a database, and if those rules are at least one element of the data we are inserting the database aborts the transaction. Hence ACID is a set of properties of database transactions intended to guarantee data validity despite errors, power failures, and other mishaps. [36]

Core Metadata Fields

Job Sector: While present across all job portals, the definition of ‘sector’ was different for each site and even different on the same site, making it a column filled with subjective and inconsistent values. For example: the job "Senior Software Engineer" was categorized in the sector "IT Services and IT Consulting", and in the same site (kariera.gr) the job title "Senior Software Engineer" was categorized in the sector "Technology, Information and Internet, Software Development, and IT Services and IT Consulting". As we can derive from the previous example it is a field that needs information to be usable for any data analysis, however for this thesis this field could not contribute even if it was normalized, because the analysis focus on the IT sector, which means that almost all the data used are from this field, it is true that there are job posts from other sectors as well, mainly scraped at the testing-developing phase. This data, however, does not affect the data analysis due to the methods used, which in all cases are pinpointing the job posts which are included in the analysis either from the title or from the job post content in the job description, as we will see more specifically later.

Site: This field records the origin platform of the job post, this field is quite important from the technical aspect but also for a part of the data analysis, especially at the testing phase. From a technical aspect it was an important field because there was optimization and troubleshooting situations where this field was pivotal e.g. when normalizing the date (which will be analyzed in a later chapter) in order to update the dates with the normalized format the site field was necessary, because each site had different format.

URL: The complete hyperlink to the original job post, this field was stored to serve as a direct reference for validation and troubleshooting during the scraping phase.

Identification Keys (IDs)

Internal ID (Primary Key): the Primary Key named *id* which is the **Primary Key** of the database regarding the identification of each element as a row number, this column is crucial for any kind of RDBMS, placing all the elements in a consecutive order, with each element identified by a unique integer, transactional databases follow a non-negotiable rule for the id field which is that all the elements **must** have an unique id, it is not possible for an element to have a Primary Key which is not an integer including of course not being blank or null and it is not possible for one id integer to be duplicate, therefore all id's must be unique.

Source Job ID: The second ‘id’ field is the job id; the job id is the extracted unique id from the respective site of each job post. Each job portal has an identification system for the job posts that they host, this became obvious at the testing phase, where I was investigating the scraping methods of each site. Each site has its own unique method for identifying a job post, below there is table which shows each site and its id system.

Table 2: Sites Identification Systems

	Id Sample	Pattern	URL Sample
Kariera.gr	254899	6-digit integer	/jobs/it-jobs/254899
Jobfind.gr	369189b3-5d21-4750-8ca4-1a21716f73be	36-digit string	JobAd/View/GR/Theseis_Er-gasias/369189b3-5d21-4750-8ca4-1a21716f73be
Skywalker.gr	01k7h9mjfwzfp6haz8v9b9hw69	26-digit string	el/aggelia-er-gasias/01k7h9mjfwzfp6haz8v9b9hw69
LinkedIn.com	4334725515	10-digit integer	/jobs/view/4315944269

The main reason why the job id field was very important is the fact that a duplication check was conducted with this id, and more specifically during the data insertion process²⁵. Another reason the job post id proved to be crucial was for the LinkedIn case where eventually at some point it was not possible to scrape by the search page²⁶ and the scraping process were based entirely on the job ids extracted.

There were some of the fields that other papers utilized (e.g. [15]), despite the value they would have provided if they were to be implemented this was not possible, those were:

Education Level

This field were on ‘LinkedIn.com’ as a semi-structured data field which was present almost always however in other platforms was either only in some job posts as semi-structured/Unstructured data or it was not at all mentioned. Since this field was not in some kind of form in all the sites, it was not possible to have it as a column in the dataset.

Estimated Salary

This field would be very useful for estimating the salary related to skills, products or technologies, however for the Greek job portals it is mostly not available, very few job posts have a salary, and due to that having an ‘Estimated Salary’ field was not possible.

²⁵ This process was analyzed previously at the chapter 4.1.5 “Data Insertion to the Database”

²⁶ This was analyzed previously at the chapter “LinkedIn Crawling Issue”

Main Fields

Job Title: One of the most important fields for this thesis was the *Job_Title*. The job title in almost all cases encapsulates the role of the job posts, this is why in the data analysis phase one of the processes is based on the job title for the categorization of the job post. However, this is a field with high cardinality²⁷ and it needs processing to draw valuable results, it will be further discussed later in the data analysis chapter²⁸

Job Description: Job description is a crucial field for any kind of data analysis, all the details about the job post are in this field. It is a field with unstructured data²⁹ where there is not any kind of rule or pattern that determines job posts even on the same site. The valuable information that we aim to extract is within a description that is filled with text that an approximate of 90% and above is of no use. Apart from the stop words and other words that are necessary for the expression in the description, many job posts are filled with elaborate descriptions about the company and its culture, which does not offer any objective information that aligns with the scope of this thesis. As we will see later this is one of the main reasons why applying data analysis methods without a predefined dictionary gives vague results with most of the words being either descriptive for the role without getting in the core hard skill needed for the role or being general attributes that was more emphasized in one label compared to the other, this will be thoroughly discussed on the Data Analysis chapter.

Company Name: This is a field with structured data and with little to no processing needed to get to a desired form, hence when locating the html element in the page on any site, apart from the data extraction phase the only processing that was necessary in some sites were the removal of trailing spaces or new line tags‘\n’. This field is important to have a view of the companies hiring in Greece, however it was not a field that offered much for this thesis, only as a complementary information to the general insights.

Workplace: Workplace is one of the important fields for this thesis, it was used extensively in the later stage and gave the location aspect to the research. Prior to being usable information, it was necessary for both a pre-processing stage and a normalization stage.

²⁷ High cardinality refers to columns with values that are very uncommon or unique. [34]

²⁸ 6.3.2 “Data Analysis using TF-IDF” the Label B

²⁹ Unstructured data: is information that either does not have a pre-defined data model or is not organized in a pre-defined manner. [38]

Despite being a field with semi-structured data which semantically in almost all job posts made sense, after a pre-processing stage at the extraction stage³⁰ data normalization was necessary for this field be usable since the definition of workplace was not homogenous, the normalization of the workplace field and the further processing that was implemented after will be analyzed at the 5.3 “Data Normalization” chapter.

Employment Type: This is a structured data field that is present in all job portal sites, there were a few cases where this field was not present on a job post, resulting in a few job posts being null. This field in some sites was part of the listing problem³¹, also it was a highly subjective field, where like in other fields as well the definition was different from site to site, also the wording and the language was different. Due to that it is a field that demands normalization in order to be usable, however it is not a field that offers value to a level that it would justify the time for creating a normalization process, also it was out of the scope of this research, in another implementation it would be feasible to create a normalization process of this field, which would give as insight about whether a job is full time, part time, work from home, internship etc.

Date: Date is a structured data field, and is considered one of the important fields, not so much for the insights that it is giving in relation to the other fields, but mainly because the insights that gathered from other fields are grounded on reality by having a specific time definition. It is also important because it is necessary for any future prospect of the methods this thesis is utilizing. Despite being a structured data field, every site had its own date type, and as it will be further analyzed in the next chapter, because data normalization was again necessary.

5.3 Data Normalization

In this chapter, I will analyze the data normalization strategy I followed in as many fields was possible. Mainly for the fields on ‘Workplace’ and for ‘Job_Date’ since the type was scarce and there was not a type for defining either of those, even though the data was interpretable.

Regional Normalization

³⁰ The extraction phase of the workplace is mentioned at the 4.1.1 “The List Extraction Problem” chapter

³¹ 4.1.1 “The List Extraction Problem” chapter

The field ‘workplace’ was a semi-structured data field that could be found in all job portals, however there was not a defined format for structuring this field and more importantly there was not a defined granularity of regional detail between neither of the sites scraped and not even for the data within the sites.

After investigating the data, I concluded that most of the workplaces are regions within a city or within a broader area which is better categorized by the closest city e.g. ‘suburbs of city’=>‘city’. Also, a lot of the workplace names do not have the name of the city within the string (e.g. "Tavros" - "Athens"), so this cannot be done with just a string match (e.g. SQL ‘LIKE’). Therefore, I had to find a way to categorize those on a bigger scale, and since this job is repetitive and consumes a lot of time, I do the following method:

First, updated the database with the normalized names in the values that previously had the name in Greek language or with a slight differentiation, examples for this can be found below. Next, I queried this:

```
SELECT count(*) as count_sec_val, workplace FROM job_posts_1 group by workplace order by count_sec_val desc;
```

Which outputs all the distinct names of the regions/cities and their count number, I copied this list, gave it to ChatGPT and asked it to give the normalized names in a JSON format. The file Regional_Normalisation JSON is the output, and I changed only some regions in Attika which are very far from Athens (e.g. "Paiania" does not categorize as Athens, but is categorized as Paiania), hence categorizes them as Athens would output biased results towards Athens.

5.3.1 Regional Normalization

Here it is important to indicate that there were three kinds of differences, naming differences, language differences, and district differences, and these differences, as I mentioned before can be found even in the same job portal. It is not possible to have valuable meaningful results if the workplace field is not normalized, this is depicted clearly if we see the results of a query in the database to print the unique workplaces and their counts, as we can see below:

```
SELECT count(*) as count_sec_val, workplace FROM job_posts_1 where workplace like '%Attiki%' group by workplace;
```

Below there is the result of the query which depicts this inference:

383 Athens, Attiki, Greece
 4 Koropi, Attiki, Greece
 3 Metamorfosi, Attiki, Greece
 8 Moschato-Tavros, Attiki, Greece
 2 Nea Erythraia, Attiki, Greece
 36 Marousi, Attiki, Greece
 35 Piraeus, Attiki, Greece
 1 Palaión Fáliron, Attiki, Greece
 2 Agios Dimitrios, Attiki, Greece
 2 Markopoulo Mesogaías, Attiki, Greece
 1 Pallíni, Attiki, Greece
 1 Vrilissia, Attiki, Greece
 1 Asprópirgos, Attiki, Greece
 1 Agios Ioannis Rentis, Attiki, Greece
 6 Gerakas, Attiki, Greece
 1 Irakleio, Attiki, Greece
 3 Alimos, Attiki, Greece
 8 Kifisiá, Attiki, Greece
 8 Central Athens, Attiki, Greece
 3 Glifádha, Attiki, Greece
 1 Spata, Attiki, Greece
 1 Moskháton, Attiki, Greece
 6 Néa Smýrni, Attiki, Greece
 3 Néa Ionía, Attiki, Greece
 1 Voúla, Attiki, Greece
 4 Attiki, Greece
 14 Khalándrion, Attiki, Greece
 15 Paiania, Attiki, Greece
 2 Aiyaleo, Attiki, Greece
 2 Neo Psychiko, Attiki, Greece
 558 **SUM OF THE ABOVE**

Therefore, I replaced all the above with Athens, since it is a keyword that is within the scope and already represents a big amount of the dataset. However, we must consider that some regions are not exactly characterized from the city “Athens” but the suburbs of the city, those are a small amount and consequently a tradeoff.

Below is an example of how I normalized the workplace field for the city “Athens” with a query, for the data that was already in the database:

Query	Query History
1 BEGIN;	
2 UPDATE job_posts_1	
3 SET workplace = 'Athens'	
4 WHERE workplace IN ('Piraeus', 'Ναυός Αττικής', 'Αθήνα', 'Marousi', 'Athens Metropolitan Area', 'Kallithea', 'Alimos'	
5 , 'Tavros', 'Kifisia', 'Chalandri', 'Peristeri', 'Δήμος Μεταμορφώσεως', 'Gerakas', 'Agios Ioannis Rentis'	
6 , 'Δήμος Παλαιού Φαλήρου', 'Δήμος Αμαρουσίου', 'Περιφέρεια Αττικής', 'Δήμος Δάφνης-Υμηττού', 'Glyfada', 'Argyroupoli'	
7 , 'Neo Psychiko', 'Αχαρνές', 'Κεντρικός Τομέας Αθηνών', 'Acharnes', 'Δήμος Αθηναίων', 'Metamorfosi', 'Γέρακας', 'Nea Filadelfeia');	

Data Output	Messages	Notifications
UPDATE 244		
Query returned successfully in 265 msec.		

5.3.2 Regional Normalization in the Data Insertion Process

As mentioned, the above query was aiming for the data that was already in the database. For the data that were inserted after the beginning of the regional normalization implementation, the normalization process was added to the data insertion part, this chapter analyses why and how it was implemented.

After considering the method of regional normalization, I concluded that I should add a function in the existing script that inserts the data into the database³², so the data are directly normalized **BEFORE** they are inserted in the database. In that way the process of normalization fits right into the existing processes, and whenever a string occurs that has not been stumbled upon before, the process stops and the data are not inserted in the database while prompted to first update the JSON of Regional_Normalisation.json and then run the script again. Only when all the location data in the JSON (which is to be inserted in the database) are normalized and they exist in the Regional_Normalisation.json and the Locations_Mapping.json³³, can be inserted in the database. Below there are two pictures of the mentioned JSON files to better understand the normalization process:

```
146  "Patra": [  
147    "Patra",  
148    "Patras, Western Greece, Greece",  
149    "ΠΑΤΡΑ",  
150    "Πάτρα",  
151    "Δήμος Πατρέων",  
152    "ΑΡΧΕΛΑΟΥ 5 | ΠΑΤΡΑ"  
153  ],  
154  "Heraklion": [  
155    "Heraklion",  
156    "Iraklion, Crete, Greece",  
157    "ΗΡΑΚΛΕΙΟ",  
158    "Νομός Ηρακλείου",  
159    "Δήμος Μαλεβιζίου",  
160    "Δήμος Χερσονήσου",  
161    "Chersonisos",  
162    "Gazi",  
163    "Tylosos",  
164    "Nea Alikarnassos"  
165  ],  
166  "Larisa": [  
167    "Larisa",  
168    "ΛΑΡΙΣΑ",  
169    "Lárisa, Thessalia, Greece"  
170  ],  
171  "Ioannina": [  
172    "Katsikás, Epirus, Greece",  
173    "Ioannina"
```

Picture 11: Regional_Normalisation.json

³² This process was analyzed above at the chapter 4.1.4 "Data Insertion to the Database"

³³ JSON with the locations latitude and longitude.

```

60      "Athens": [37.9838, 23.7275],
61      "Mesolongi": [38.3706, 21.4297],
62      "Xanthi": [41.1414, 24.8836],
63      "Nicosia": [35.1856, 33.3823],
64      "Netherlands": [52.1326, 5.2913],
65      "Karditsa": [39.3656, 21.9216],
66      "Giannitsa": [40.7919, 22.4075],
67      "Kalymnos": [36.9500, 26.9833],
68      "Korinthos": [37.9400, 22.9444],

```

Picture 12: Locations_Mapping.json

Insertion of a New Category (Key) to the Normalization JSON

The new workplace string can be inserted into the JSON either into an existing list of strings/'VALUES' that connects to one regional 'KEY' or make a new category/'KEY' and insert the string in the list of that new category. Below those two cases are explained:

1. **New name of an existing category:** In that case the job post has a workplace that already exists in the Regional_Normalisation.json as a category, and semantically the category and the new workplace name are the same, however the format of that new value is different resulting the algorithm not matching the two. For example, the category can be 'Athens' and the new workplace string of a job post could be "Athens, Attiki, Greece", if this string is not on already in the list of the category 'Athens' then they will not match, and the algorithm will prompt the user to insert that value in a category, in that case in 'Athens' and not in a new category.
2. **New category for a new name:** This case is about a workplace that is completely new and is not in Regional_Normalisation.json as the string itself and as a semantic meaning.

So, to conclude the structure of the regional normalization is the following:

Extracted Data Field > First Data Cleaning > Storage in JSON > JSON_TO_DB > Check if the data is already in the normalization file json > A) If it is the proceeds with the conversion of data to the normalized data

B) If it is not, you are prompted to first insert the new data into the normalization field and repeat the process.

5.3.3 Date Format Normalization

The date format transformation, this transformation was necessary because each site had its own date format, so 4 different formats which are the following:

Skywalker	"%d-%m-%Y"
Kariera	"%d %b %Y, %H:%M"
Jobfind	"%d/%m/%Y"
Linkedin	"%Y-%m-%d"

Here we can see every job date for every site

Since the conversion as a process is the same for all four of the ‘Spiders’ but the only thing changes is the format of the respective job portal, I decided to make that process an external utility with a function where each of the ‘Spiders’ would use it, giving as an inputs the date format and the date, and returning as an output a common format. The format I choose was the standard ISO 8601 which is “YYYY-MM-DD”.

1) To tackle problem one, this is the generalized solution:

```
from datetime import datetime
old_date="15/11/2025"
old_date_format="%d/%m/%Y"
new_date=datetime.strptime(old_date, old_date_format).date()
new_date.strftime("%Y-%m-%d")
'2025-11-15'
```

2) This problem was solved by creating a shared utility module which is placed this folder:
/exprmnts_with_scrapy/exprmnts_with_scrapy/utlis

In this folder the convert_date_format.py script was created which takes two arguments, the date string and the old date format and returns the date in new format, if the date string is not in the old format or it is any other value, it returns the value unprocessed.

6 Data Analysis

In this thesis two main data analysis methods were applied, TF-IDF and Dictionary-Based Entity Extraction.

6.1 Pre-processing and Data Cleaning

The pre-processing stage of the TF-IDF and Dictionary-Based Entity Extraction methods had two similar stages, which is the word split³⁴, and the punctuation removal (which had differentiations) apart from those there are no other similarities, this is due to the different kind of processing each method is conducting. For the TF-IDF method the data of the input must be in a specific format to output valuable results, which is a text description stripped from punctuations, stop words, links and any other non-word characters, only words separated by spaces.

Whereas in Dictionary-Based Entity Extraction method the data cleaning is limited, because the item identification inside the dataset is done by regex string matching, so the algorithm is searching for specific strings in the text and ignores other text, the only pre-requisite data cleaning it needs to work accurate is the removal of dots ('.'), commas (','), and parenthesis ('()') because if those characters are next to the strings we search they will not be matched, the words must consists of the exact same characters between spaces that were set in regex or else they will not be found.

Below is an overview of the TF-IDF process:

1. Links Removal: `text = re.sub('https?://S+|www\.\S+', '', text)`
2. Punctuation Removal
3. Split separated words
4. Lowercase
5. Stop words removal: Both Greek and English

And here is an overview of the Dictionary-Based Entity Extraction:

³⁴ This was analyzed at 4.1.4 “The List Extraction Issue”

1. Punctuation Removal [.,()]
2. Split separated words
3. Lowercase

6.2 Data Analysis Environments

In this chapter, I will analyze the two distinct environments in which different analysis was conducted.

6.2.1 Spark in a Virtual Machine

One of the two environments developed is Spark which was integrated inside a ‘*Virtual Box*’ Virtual Machine (VM), the VM was given at the ‘Big Data’ course by Dr. Akritidis and had a functional and configured Hadoop and Spark environment. However, there was some necessary extra technical configuration to be integrated with the database of the application which will be analyzed in this chapter.

Spark Environment Configuration and Data Integration

To perform robust data analysis, a Spark environment was configured within a Linux Virtual Machine (VM) to establish a seamless connection with a PostgreSQL database. This integration serves as the primary data ingestion layer for the application.

Database Connectivity via JDBC

The communication between Spark and the PostgreSQL instance was implemented using the Spark SQL DataFrameReader interface. By utilizing the `.read.format("jdbc")` method, the system establishes a connection to the external relational database.

```
df_job_posts = sc.read.format('jdbc').options(  
    url='jdbc:postgresql://10.0.2.2:5434/jobapplication',  
    dbtable='job_posts_1',  
    user='postgres',  
    password='[REDACTED]').load()
```

Picture 13: The DataFrameReader configuration with the database

As illustrated in Picture [13], the connection string utilizes the IP address 10.0.2.2. Within the Virtual Box networking architecture, this address represents the default NAT gateway, allowing the guest VM to access services hosted on the host machine.

Driver Integration and Runtime Setup

Beyond the configuration of connection strings, the environment requires the PostgreSQL JDBC driver to be present in the Spark class path. The driver (JAR file) was found from

the official PostgreSQL repository and deployed to the `$SPARK_HOME/jars/` directory. This ensures that the Spark runtime can dynamically load the necessary classes for protocol translation during execution.

Modular Development Workflow

The development process followed a modular "cell-based" architecture within the notebook environment. By isolating discrete stages of the process into independent execution units:

- **Granular Debugging:** Errors can be isolated to specific transformation steps.
- **Data Validation:** Intermediate DataFrames can be inspected for schema integrity and data quality before proceeding to downstream algorithms.
- **Efficiency:** Allows for iterative development without re-running the entire ingestion pipeline.

6.2.2 Environment Isolation: Extraction vs. Data Analysis

To ensure modularity and system stability, the project architecture was divided into two distinct execution environments. This separation was necessary to manage disparate library dependencies and prevent version conflicts between the data acquisition and data processing stages.

The Data Extraction Environment (venv)

This environment was dedicated exclusively to web scraping and data ingestion. By isolating the extraction scripts, the system ensured that network-intensive tasks and their specific driver requirements (e.g., Scrapy) did not interfere libraries and dependencies of the data analysis environment. All the above ETL (Extract, Transform, Load) processes mentioned are conducted within this virtual environment called 'venv'.

The Data Analysis Environment (ml-venv)

The ml-venv served as the primary Data Analysis Environment. While it functioned as an experimental sandbox for testing, it also transitioned into a production-ready environment for the project's core analytical deliverables.

Specifically, this environment was used to implement and execute the TF-IDF (Term Frequency-Inverse Document Frequency) algorithms. Because the results generated by these TF-IDF models constitute a primary finding of this thesis, the ml-venv was

maintained as an environment where analytical experiments could be directly converted into final research outputs.

To support the end-to-end analytical pipeline, a comprehensive suite of Python libraries was integrated into this environment. The main libraries installed were of course pip as a package manager, Jupyter Notebook, Pandas and NumPy for data manipulation, Scikit-learn for the implementation of the TF-IDF vectorization and modeling, and NLTK (Natural Language Toolkit) for text preprocessing. Furthermore, the environment was equipped Psycopg to facilitate direct communication with the PostgreSQL database.

Elasticsearch and Kibana

In this chapter, I will analyze how I utilize Elasticsearch and Kibana in the Thesis. Elasticsearch has two main functionalities in this Thesis:

- A) Data analytics tool in most cases combined with other tools and methods
- B) Using it as a verification on results of other methods

Elasticsearch

Elasticsearch “is a source-available search engine developed by Elastic. It is based on Apache Lucene and provides a distributed, multitenant-capable full-text search engine, it stores data as schemaless JSON documents, making it ideal for full-text search, logging, security analytics, and AI applications” [23] [24]

Some fundamental components of elasticsearch are the following:

- **Index:** Index essentially, we can say that it is a ‘table’ in elasticsearch, more specifically: “An index is a fundamental unit of storage in Elasticsearch. It is a collection of documents uniquely identified by a name” [25]
- **Document:** Documents in Elasticsearch could also be parallelized with the ‘rows’ of a table, in this case the rows of an index, more specifically: “Elasticsearch serializes and stores data in the form of JSON documents. A document is a set of fields, which are key-value pairs that contain your data. Each document has a unique ID, which you can create or have Elasticsearch auto-generate.” [25]

Elasticsearch as a Data Analytics Tool

Elasticsearch is versatile platform which gives a plethora of valuable tools, and some of them available with no further configuration apart from the mapping. In the thesis elasticsearch tools give a unique perspective to the dataset, from field statistics to placing the

dataset on the map making almost every field of it queryable. In this chapter we will investigate those tools, and especially the map features.

Configuration of Elasticsearch in Docker

To provide efficient indexing and real-time visualization of the analyzed job data, the Elastic Stack (ELK) was deployed using containerized architecture. This approach ensures that the search engine and its corresponding dashboard are decoupled from the host operating system, facilitating a reproducible and scalable environment.

Container Orchestration via Docker Compose

The deployment was managed through **Docker Compose**, utilizing a YAML-based configuration to orchestrate two primary services: **Elasticsearch** (v8.15.0) and **Kibana** (v8.15.0).

- **Elasticsearch Node:** Configured as a single-node discovery type to optimize local resource utilization. To ensure data persistence across container restarts, a dedicated Docker volume (es_data) was mapped to the container's internal storage path. Memory limits were explicitly defined via ES_JAVA_OPTS to stabilize the JVM heap size at 1GB.
- **Kibana Integration:** Deployed as the visualization layer, Kibana was configured to communicate with the Elasticsearch service over a dedicated bridge network (esnet). The depends_on directive was implemented to ensure a logical boot sequence, where the visualization interface only initializes after the search engine service is healthy.

Below is the docker-compose.yml (Elasticsearch + Kibana) file, this was instantiated by hitting *'docker compose up'* in the terminal.

```
• version: "3.9"
•
• services:
•   elasticsearch:
•     image: docker.elastic.co/elasticsearch/elasticsearch:8.15.0
•     container_name: elasticsearch
•     environment:
•       - discovery.type=single-node
•       - ES_JAVA_OPTS=-Xms1g -Xmx1g
•       - xpack.security.enabled=false # Disable security for simplicity
•       - xpack.security.http.ssl.enabled=false
•     ports:
```

```

•     - "9200:9200"
•     - "9300:9300"
•     volumes:
•     - es_data:/usr/share/elasticsearch/data
•     networks:
•     - esnet
•
•     kibana:
•     image: docker.elastic.co/kibana/kibana:8.15.0
•     container_name: kibana
•     environment:
•     - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
•     - XPACK_SECURITY_ENABLED=false
•     ports:
•     - "5601:5601"
•     networks:
•     - esnet
•     depends_on:
•     - elasticsearch
•
•     volumes:
•     es_data:
•
•     networks:
•     esnet:
•

```

After instantiating elasticsearch in a docker container, the next step was to insert data from PostgreSQL database. Below are the steps to intergrade Elasticsearch and PostgreSQL with Python:

1. Install elasticsearch python in the venv³⁵: *pip install elasticsearch*
2. Install pycopg library³⁶
3. Instantiate Elasticsearch object in the python script and connect to elasticsearch:

```

21     es=Elasticsearch(
22         "https://6a0a15b448304663af13cf5274f982c0.us-central1.gcp.cloud.es.io:443",
23         basic_auth=("elastic", "XXXXXXXXXXXXXXXXXXXX"),
24         request_timeout=30,
25     )

```

³⁵ The venv has the same name 'venv'

³⁶ The exact steps for installing this dependency are in chapter 4.1.5 'Data Insertion to the Database'

4. Create the data to be inserted in the Elasticsearch in dictionary format:

```
58 doc = {
59     'id': row[0],
60     'job_id': row[1],
61     'job_title': row[2],
62     'job_company': row[3],
63     'workplace': workplace,
64     'employment_type': row[5],
65     'job_sector': row[6],
66     'job_date': row[7],
67     'job_description': row[8],
68     'site': row[9],
69     'url': row[10],
70     # This is the location field which in Kibana is instantiated as geo_point
71     'location': {
72         "lat": lat,
73         "lon": lon
74     }
75 }
```

5. Insert the data in the elasticsearch with the `es.index()` command:

```
77 response = es.index(index="job_ads", id=row[0], document=doc)
78 print(response["result"])
79 print(response["_index"])
```

Mapping Fields in Elasticsearch

Mapping in Elasticsearch on a high level is the definition of fields in a dataset, it is very important for datasets that have fields with text and fundamental for fields like Geolocation and date. More specifically “Mapping is the process of defining how a document³⁷ and the fields it contains are stored and indexed. Each document is a collection of fields, which each has their own data type. When mapping your data, you create a mapping definition, which contains a list of fields that are pertinent to the document.” [26]

I concluded with the following mapping for the dataset³⁸:

```
• PUT /job_ads
• {
•     "mappings" : {
•         "properties" : {
•             "id" : {
•                 "type" : "long"
•             },
•             "job_id" : {
•                 "type": "text",
•                 "fields": {
•                     "keyword": {
•                         "type": "keyword",
•                         "ignore_above": 256
•                     }
•                 }
•             }
•         }
•     }
```

³⁷ document is the dataset

³⁸ For the creation of the elasticsearch mappings, this tutorial was helpful [39]

```

    }
  },
  "job_title" : {
    "type": "text",
    "fields": {
      "keyword": {
        "type": "keyword",
        "ignore_above": 256
      }
    }
  },
  "job_company" : {
    "type": "text",
    "fields": {
      "keyword": {
        "type": "keyword",
        "ignore_above": 256
      }
    }
  },
  "workplace" : {
    "type": "text",
    "fields": {
      "keyword": {
        "type": "keyword",
        "ignore_above": 256
      }
    }
  },
  "employment_type" : {
    "type": "text",
    "fields": {
      "keyword": {
        "type": "keyword",
        "ignore_above": 256
      }
    }
  },
  "job_sector" : {
    "type": "text",
    "fields": {
      "keyword": {
        "type": "keyword",
        "ignore_above": 256
      }
    }
  },
  "job_date" : {

```



```

    •         "type" : "date"
    •     },
    •     "job_description" : {
    •         "type": "text",
    •         "fields": {
    •             "keyword": {
    •                 "type": "keyword",
    •                 "ignore_above": 256
    •             }
    •         }
    •     },
    •     "site" : {
    •         "type" : "keyword"
    •     },
    •     "url" : {
    •         "type": "text",
    •         "fields": {
    •             "keyword": {
    •                 "type": "keyword",
    •                 "ignore_above": 256
    •             }
    •         }
    •     },
    •     "location": {
    •         "type": "geo_point"
    •     }
    • }
    • }

```

Geolocation In Elasticsearch

Geolocation in elasticsearch needs two things to work at a first level:

1. Instantiating the mapping of an index needs a field with specific location type, in this implementation I used `geo_point` ("type": "geo_point" Note the table above), however there are other 'field' types in elasticsearch that are Geolocation related like: `geo_shape`, `point` (Cartesian point), `shape` (Cartesian shape).
2. The values to inserted in the index need to be in the specific format of the corresponding type. More specifically for `geo_point` and in other Geolocation types the latitude and longitude are necessary (See table B):

B)

```
"location": {
```

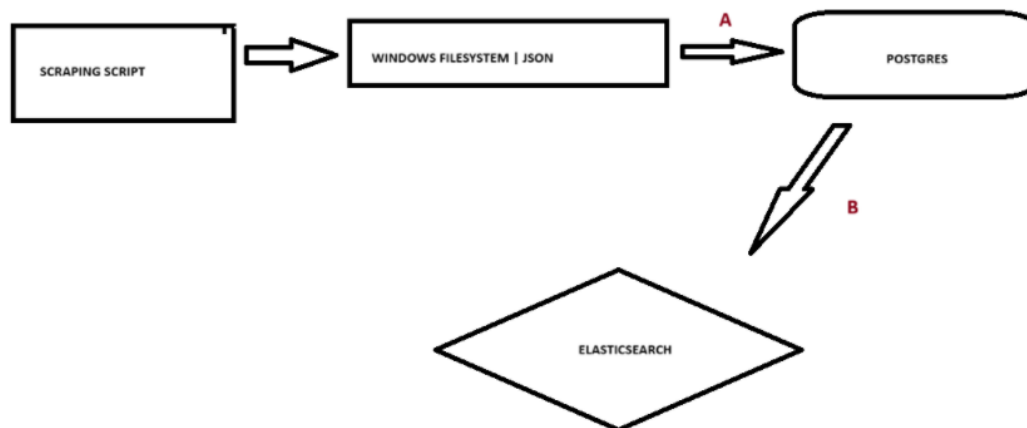
```
"lat": 37.9839,  
"lon": 23.7279}
```

Implementation of the Elasticsearch Map

I will give a brief explanation of the implementation of the elasticsearch map, here we will see how the things that were analyzed above, like regional normalization³⁹ and the geolocation are combined to create the map feature.

1. Normalization of all workplaces to cities
2. Creation of a JSON with every city and its coordinates
3. Creation of elasticsearch mappings which includes an extra field 'location' and its type is geo-point.
4. Run a script for the data insertion into Elasticsearch, which gets for every *workplace* its corresponding latitude and longitude.

Below there is an image depicting where Elasticsearch is in the process and where the insertion and the transformation of the data is conducted.



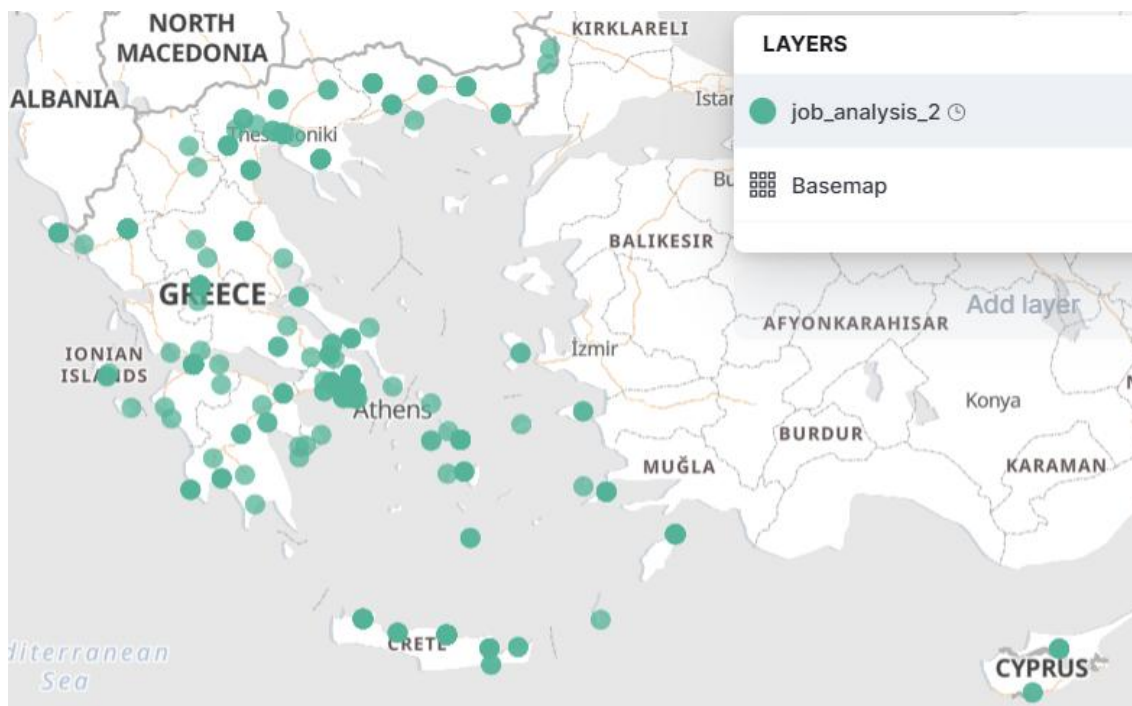
Picture 14: Data integration with Elasticsearch

A: This is where the normalization is being conducted at the import of the data to the database

B: This is where the import to the elasticsearch is being done, this is also where the coordination ('location' field) are added based on the city of the normalized 'workplace'

The picture below shows the end result, the dataset placed on a map:

³⁹ The Chapter 5.3.1 "Regional Normalisation"



Picture 15: The dataset on the map

Kibana Query Language

Kibana Query Language or KQL, is a SQL like language of Kibana with which queries are conducted to the data. As per Elastic's site, "The Kibana Query Language (KQL) is a simple text-based query language for filtering data. KQL only filters data, and has no role in aggregating, transforming, or sorting data. Use KQL to filter documents where a value for a field exists, matches a given value, or is within a given range." [27]

Other Kibana Tools

Kibana offers some statistics tools by default called Field Statistics⁴⁰ which help in giving a quick overview of the dataset, it is a great way to understand the data quality and possibly patterns forming. The most useful feature of this tool, however, is that it can be used for custom KQL queries which means that it is possible to investigate specific portions of the dataset by filtering the columns of the dataset with custom criteria. This feature

⁴⁰ "The Field statistics view in Discover provides statistical summaries and visualizations of your data fields. View distributions, top values, minimum and maximum ranges, and cardinality to quickly understand your data quality and patterns. Use field statistics to identify data issues, understand field characteristics, and discover insights before building visualizations." [40]

was used extensively for troubleshooting and as part of the data analysis. Below we can see an example of the field statistics:

Documents (3,158)

Field statistics

Type	Name	Documents (%)	Distinct values	Distributions	Actions
k	job_description.keyword	36 (1.14%)	35	top 10 of 35 categories	
f	job_id	1,000 (100%)	997		
k	job_id.keyword	3,158 (100%)	3106	top 10 of 3106 categories	
f	job_sector	1,000 (100%)	134		
k	job_sector.keyword	3,158 (100%)	219	top 10 of 219 categories	
f	job_title	1,000 (100%)	756		
k	job_title.keyword	3,158 (100%)	1986	top 10 of 1986 categories	
	location	3,158 (100%)	106		
k	site	3,158 (100%)	4	4 categories	
f	url	1,000 (100%)	997		
k	url.keyword	3,135 (99.27%)	3080	top 10 of 3080 categories	
f	workplace	1,000 (100%)	56		
k	workplace.keyword	3,158 (100%)	107	top 10 of 107 categories	

Picture 16: Example of the Field Statistics

Concrete uses of Field Statistics will be presented in the following chapters, and in the results of the thesis.

6.3 Data Analysis Methodologies

In this Chapter I will analyze the two Methodologies followed, which are the ‘Data Analysis using TF-IDF’ and ‘Dictionary-Based Entity Extraction’.

6.3.1 TF-IDF Algorithm

Here I will analyze the TF-IDF method in the Spark VM environment. I used TF-IDF in two ways, first of course as a data analysis method aiming to find insights about the dataset and second as a correction tool aiming to find parsing errors like the one mentioned above⁴¹ and ‘noise’ words which include some stop words which I stumbled upon after the first results.

Term Frequency-Inverse Document Frequency (TF-IDF) is a foundational statistical technique in natural language processing and information retrieval designed to quantify the importance of a given word within a specific document relative to a larger corpus. At a high level, it transforms unstructured raw text into a meaningful numerical

⁴¹ Especially the ‘Concatenated Words Scraping Issue’

representation by balancing two distinct metrics: the presence of a term within a local document (Term Frequency) and how rare it is across the entire global dataset (Inverse Document Frequency). By computing the product of these two components, the algorithm penalizes highly ubiquitous words, such as common articles or prepositions, while mathematically elevating the domain-specific vocabulary that uniquely characterizes the text. Consequently, this weighting mechanism generates sparse numerical vectors that capture the distinct lexical footprint of a document, serving as an essential preprocessing step for downstream machine learning tasks such as text classification, topic modeling, and discriminative feature analysis.

About the TF-IDF algorithms that were built in the thesis, more specifically they do Discriminative Feature Analysis using TF-IDF and Logistic Regression, overall, 4 main algorithms were built, each of them has one of the decisive parameters changed. The parameters and the algorithms will be discussed more thoroughly in the next chapters.

6.3.2 Data Analysis using TF-IDF

Generally, the TF-IDF algorithms developed focus on two main parameters; those two parameters are the n-gram and the labels. The n-gram parameter is about the length of the tokens that the TF-IDF gets as an input, hence the definition of the length of words the TF-IDF algorithm counts as tokens. The TF-IDF tokens can be one-word, two-words or more, in the implementation of this thesis I used one-word and two-word n-grams, the results of each are different and the results in the two-gram approach are of greater quality than the one gram. This will be further analyzed by reviewing the results in a later chapter. The labels parameter is about how many different distinct categories of items are calculated. More specifically, TF-IDF is using labels to discover the uniqueness of the words, so word tokens get scored relative to their presence against other words in the same label and in other labels. Generally, we can say that a high TF-IDF score is reached by a word that has high frequency in a specific document but a low frequency in the rest of the documents, resulting in being representative about the document that is appears with high frequency.

The four different TF-IDF algorithms that occurred from adjusting those parameters mentioned are: one-gram with two labels, one-gram with four labels, two-grams with two labels, and two-grams with four labels.

Another decisive configuration for TF-IDF is the selection of the field that will be set as label, which in our case is the selection of which column our data will be labeled. Decisive of course is also the choice of the text data input that we give to the algorithm; however, in our case the job description is a clear choice for all implementations. In this thesis the dataset has 11 columns including the job description. This means that technically we could have 10 different kinds of labels, but practically some columns would not give meaningful results, for example the date column is not a good choice for a label.

In this Thesis I used two different categories of labels and was applied to all four of the different implementations of the TF-IDF, I will outline those different categories below, however the category B will be further analyzed because it is a label that was achieved programmatically and it was not directly available like category A:

- A) Column 'Site': Where the labels were the sites from which the data were scraped: LinkedIn.com, kariera.gr, skywalker.gr, jobfind.gr
- B) Column 'Job Title': Here the labels are the job titles that were derived programmatically: Software Engineer, Data Engineer, Support Engineer, and IT (Or IT Systems Trainer and Support⁴²)

Label A is a column with Low cardinality⁴³ because there are only four possible values that a job posts can have, hence in this dataset one job post can only be from one site. This label was used at first to test the TF-IDF algorithm and not necessarily to find valuable features because the differences between 'sites' does not give an insight which could describe a certain trend or 'uniqueness' that can be useful, for example the algorithm indeed found the unique features of jobfind.gr which was mainly words in Greek since most of the job posts were in Greek, however this is not a uniqueness that tell us anything about the technology sector rather than which and what kind of companies post to each site, which is out of scope for this thesis. However, unexpectedly it was proven to be quite useful for finding stop words, concatenated words⁴⁴ and other errors, that's why I use this label through the testing process, and some examples of the results will be presented in the next chapters.

⁴² This was a job post which was present in many different places

⁴³ Low-cardinality refers to columns with few unique values. [34]

⁴⁴ Due to the concatenated words issue that was mentioned above.

Label B was derived programmatically because the job title is a column with high cardinality even though it is a column that has categories (job roles) which are more discrete semantically. More specifically, there are roles that are semantically the same, but the wordings are slightly different, in many cases there were job posts with roles that were the same, but the job titles had wordings with differences. For example, the title ‘Software Engineer - Software Developer’ and ‘Medium/Senior Software Engineer (Java)’ is the same role. Of course, each job post has its unique features, but when jobs of the same role are aggregated into one category and compared with other roles, we investigate what are the distinguishable features of each role/category.

Results of the TF-IDF Data Analysis

‘Site’ Label, Two Labels, One-gram – Testing Phase

Here is the first implementation of the TF-IDF with column ‘Site’ as label. As mentioned above, this was mainly for testing and optimization purposes. It is important to point out that those were the first results, so there are errors which were corrected, like the examples that we will see below. The two sites configured as labels are the kariera.gr and linkedin.com as we can see below, where linkedin.com is the label 0 and kariera.gr is the label 1.

Now, the TF-IDF core process starts
 THIS ALGORITHM WORKS LIKE THIS: You must have a specific category to train it, not just from the words.
 For example, `#le.fit(df_stemmed["job_sector"])`, train the algorithm to track WORDS corresponding to JOB_SECTOR

```
### Label Encoding

from pyspark.ml.feature import StringIndexer

label_indexer = StringIndexer(
    inputCol="site",
    outputCol="label",
    handleInvalid="error"
)

df_li = label_indexer.fit(df_TF_IDF).transform(df_TF_IDF)

df_li.select("label", "site").distinct().show()
#df_li.take(4)
```

label	site
0.0	linkedin.com
1.0	kariera.gr

The results were the following:

	label_0_words	label_1_words		label_0_words	label_1_words
0	ukin	validata	15	amount	systemat
1	uninterrupt	cf	16	datacent	miss
2	wfh	sql experi	17	claim	shortlist
3	town	retrain	18	bugzilla	saturday
4	informat	complementari	19	password	drift
5	broker	zs	20	behalf	vibe
6	operation	prefect	21	recent	ne
7	vms	synergi	22	συμβόλαιο	daycar
8	ισχυρό	gradl	23	offboard	toddler

9	εκδηλώσεις	introduc	24	jqueri	letter
10	assum	kept	25	thank	jurisdict
11	remov	pmi	26	civil	kariera
12	incl	ka	27	add	ia
13	outlet	openshift	28	divid	fantast
14	flagship	bamboo	29	alon	sa

TF-IDF Corrections and Troubleshooting

As we can see from the results there are some words that are two concatenated words, this is due to the issue that was mentioned above⁴⁵, there are some words that have not been separated from the separator algorithm⁴⁶, because they have a different pattern. For example, the ‘ukin’ which originally before the lower-casing is ‘UKIn’ that consists of ‘UK’ and ‘In’.

equivalent to those stated for the UKIn addition,

This was not separated because there are three upper case letters next to each other which is passing from the separator algorithm, there is no rule that would detect this pattern apart from an algorithm that separates words that can be found on the English dictionary. However, cases such as this one were not frequent enough to justify such a solution, that’s why a simpler solution was adopted, correcting such cases with SQL:

```
update job_posts_1
SET job_description = REPLACE(job_description, 'UKIn', 'UK In')
where job_description like '%UKIn%';
```

This pattern is an example of the TF-IDF utilized for finding errors and troubleshooting.

Below there are other examples of such corrections:

Another example is the ‘SQLExperience’

⁴⁵ Chapter 4.1.3 Concatenated Words Issue

⁴⁶ The Separator algorithm can be found in chapter 4.1.3 “Concatenated Words Scraping Issue”

7	facet	dba	22	sportsbook	bms
8	cbtalent	saturday	23	σύγχρονη	prefect
9	cicd	biopharmaceut	24	άνθρωποι	ethernet
10	alon	male	25	impecc	ek
11	uml	ka	26	σύντομη	ne
12	maven	reus	27	οργανισμός	persever
13	informat	shortlist	28	pytest	generalist
14	behalf	giac	29	βιομηχανία	li

TF-IDF Corrections and Troubleshooting – 4 Sites Label

Below we can see the labels as they are outputted from the ‘Label Coding’ stage. The labels numbers represent the quantity of each category, so in this example the site with the most job posts is LinkedIn.com and the site with the minimum number of job posts is Skywalker.gr.

	Linkedin.com	Kariera.gr	Jobfind.gr	Skywalker.gr
0	wfh	validata	trenkwald	staf
1	cbtalent	saturday	absolut	dear
2	freak	kariera	ζητείται	παλαιό
3	υπόψη	daycar	ζητάει	επιθυμούν
4	els	toddler	cypriot	el
5	liter	kept	placement	αντικείμενα
6	vet	δυνατοτήτων	oci	titl
7	massiv	gui	temporari	αναφέροντας
8	δεικτών	brds	eb	χρήστη
9	gr	male	κινητά	χρησιμοποιεί
10	behalf	dpo	amp	γίνουν
11	vms	zs	καλαμαριά	ώρα
12	γένει	kaggl	προϊόν	wpf
13	θέμα	παροχες	εκτύπωσης	ευθύνες
14	εργοδότης	cf	ενεργή	linen
15	recent	απαραίτητο	intermedi	αυτόν
16	αρχιτεκτονική	sehr	adecco	santorini
17	brows	authoris	συστηματική	αναπτυσσόμενες
18	ado	βέλτιστων	normal	metaxa
19	ισχυρό	customis	fascin	μονάδων
20	warranti	sigmasoft	ζητά	καινοτόμων
21	add	modbus	ερωτήσεις	υλοποιήσεις
22	poc	sheet	gradual	outreach
23	soon	renumer	pharmaci	είτε
24	σχετικού	αυτός	netherland	τεχαν
25	coo	pitch	chrome	εργαστούν
26	βαθιά	fp	τεχνολογικό	unforgett
27	alon	πληροφοριακού	αττική	flair
28	pickup	shortlist	zephyr	σημειώματα
29	acca	pack	dialog	menu

TF-IDF One-Gram Two Labels – Job Title Label

Here are the results of the one-gram and two labels TF-IDF with the job title as label, the difference is already visible with a first glance, almost all the results are relevant, however some of them offer valuable insight like databrick in ‘Data Engineer’ and embed in the ‘Software Engineer’ label:

	Software Engineer	Data Engineer		Software Engineer	Data Engineer
0	embed	big	15	hr	thorough
1	softwar	experi	16	pars	remuner
2	vacat	engin	17	multipl	fabric
3	senior	behalf	18	disrupt	method
4	safeguard	cleans	19	adher	databrick
5	name	environ	20	quot	recommend
6	proper	factori	21	updat	etl
7	map	select	22	applic	exceed
8	practition	semant	23	tailor	particular
9	testabl	degre	24	epignosi	conduct
10	resid	militari	25	numer	attract
11	robot	ssis	26	program	wareh
12	form	kong	27	websit	written
13	affin	hong	28	fund	unstructur
14	rest	raw	29	judgment	maintain

TF-IDF One-Gram Four Labels – Job Title Label

Here are the results of the one gram and four labels, below is the label description:

	'Software Engineer'	'Support Engineer'	'Data Engineer'	'IT Support'
0	τεχνολογίες	up	big	respons
1	embed	replic	cleans	bachelor
2	softwar	resolv	capac	materi
3	capac	weekend	mine	w
4	senior	l	challeng	classroom
5	environment	motto	normal	atlassian
6	work	issu	feed	liaison
7	αντίστοιχη	τηλεπικοινωνιών	workspac	similar
8	βάσεων	τμήμα	add	st
9	saturday	support	blockchain	previous
10	robot	even	unstructur	suit
11	expens	pressur	semi	updat
12	ένα	custom	tableau	nd
13	zs	διαχείρισης	quo	present
14	ka	spoken	raw	strong
15	pixel	unresolv	ssis	prefer
16	appli	kick	among	effect
17	regress	inquiri	method	prepar
18	safeguard	aris	recommend	knowledg
19	ανάπτυξης	clientel	appli	relev

20	αναζητούμε	respond	behalf	identifi
21	form	screen	extract	aim
22	ομάδας	applic	ms	task
23	timelin	fill	seminar	resolut
24	throughout	thus	etl	orient
25	stem	pcs	cornerston	structur
26	υποψήφιος	brows	roll	experi
27	rag	invit	databrick	good
28	εξοικείωση	diagnost	heavili	languag
29	anyon	upon	fabric	oper

TF-IDF Two-Gram Two Labels – Job Title Label

Here with the two grams, we can see that the results are better compared with those of the previous methods and offer a better view for all the labels. Also, it is important that there are more hard skills compared to other results.

	software engineer	data engineer		software engineer	data engineer
0	capacity senior	data engineer	25	talented software	bachelor
1	embedded software	big data	26	consumer	understand data
2	software engineer	big	27	adherence	data products
3	software	written english	28	members team	python sql
4	embedded	skills	29	web technologies	statistics
5	robotics	quality oriented	30	drive innovation	processes data
6	senior software	engineer	31	c python	informed
7	collaborate effectively	degree	32	qualifications years	data processes
8	pool	looking data	33	members across	experience data
9	zones	spark experience	34	intellectually	pipelines workflows
10	time zones	integration across	35	operator	apache spark
11	vehicles	lifecycle management	36	improvement initiatives	pipelines familiarity
12	stacks	data pipelines	37	simulation	motivated data
13	environmental	knowledge python	38	strong plus	continually
14	git version	azure databricks	39	tube	data factory
15	bias	practices data	40	infrastructure aws	experience good
16	possess	impactful data	41	world top	within data
17	work within	deliver impactful	42	five	unstructured
18	minded	ms ssis	43	agencies	driven insights
19	requirements strong	data systems	44	ensuring alignment	raw
20	senior	recommendations	45	design architecture	experience
21	json xml	part group	46	business operations	unstructured data
22	optimising	g azure	47	systems expert	data processing
23	requirements translate	plus understanding	48	aiming	time data
24	teams design	data engineers	49	graphics	strong analytical

TF-IDF Two-Gram Four Labels – Job Title Label

Here we see the results of each label are highly relevant to their category, and indeed the words extracted from the job description identify to their respective role.

	'Software Engineer'	'Support Engineer'	'Data Engineer'	'IT Support'
0	capacity senior	applicants resolve	capacity data	across greece
1	embedded software	application support	big data	responsibilities
2	software engineer	users collaborate	big	bachelor
3	robotics	teams continue	data engineer	bachelor degree
4	engineer continue	ups	engineer continue	presentation skills
5	τεχνολογίες	internal users	end data	platforms tools
6	senior software	support customers	blockchain	liaison teams
7	saturday	collaborate engineering	data processes	tools additional
8	επαγγελματικής ανάπτυξης	τηλεπικοινωνιών	mining	knowledge learning
9	αντίστοιχη θέση	τμήμα	within data	update training
10	αντίστοιχη	πνεύμα	based experience	users prepare
11	stacks	ομαδικό πνεύμα	select	tasks responsibilities
12	βάσεων δεδομένων	resolve complex	experience relevant	sector aiming
13	βάσεων	διαχείρισης	looking data	experience liaison
14	vehicles	ομαδικό	financial sector	classroom training
15	years software	τεχνικών προβλημάτων	lifecycle management	training structured
16	embedded	customers internal	solid foundations	w
17	ένα	weekends	structured unstructured	operator responsibility
18	complex systems	l	experience microsoft	application productively
19	collaborative innovative	installs	understanding etl	systems atlassian
20	πτυχίο αει	γνώση αγγλικών	continually	deliver business
21	infrastructure powers	άριστη γνώση	transformations	suite strong
22	talent lms	requirements work	industry best	preferably within
23	errors	required qualifications	stages	responsibility effectively
24	press	αναζητά	business logic	training users
25	software	αγγλικών	develop ai	resolution excellent
26	teamwork skills	άριστη	g azure	training user
27	used millions	support engineer	happen	remote training
28	personality	needs technical	experience financial	material provide
29	diagnosing	understanding networking	time data	operator netcompany

6.3.3 Adapting SARO for Dictionary-Based Entity Extraction

‘Ontology-Guided Job Demand Analysis’ [2], serves as the foundational information representation that guides an Ontology-Based Information Extraction (OBIE) pipeline to automatically identify, extract, and classify skills within unstructured job advertisements. While the methodology implemented in this thesis draws heavy inspiration from OBIE and the structural foundation of SARO, it diverges to optimize frequency and co-occurrence analysis.

Rather than deploying a strict OBIE system which relies on deep semantic graph traversals and inference rules, this pipeline utilizes Dictionary-Based Entity Extraction. By

flattening the native semantic graph into a customized JSON format, the original ontology was adapted into a targeted gazetteer, acting as a controlled vocabulary tailored for this specific environment. This practical derivative of SARO focus purely on entity nodes, enabling computationally efficient, explicit extraction of programming languages and databases from the dataset.

6.3.4 Dictionary-based Entity Extraction Method Implementation

In this Chapter will be analyzed how a lookup dictionary from SARO Ontology was created and how the method of Gazetteer-Based Extraction was Implemented.

Pre-Processing Stage | Configuration of the Mapping JSON

SARO Ontology can be found in GitHub Repository [28], the ontology file is a Turtle (.ttl) file⁴⁷ which was not a suitable format for the environment and the processing that was developed, that's why it was converted it into JSON key-value pairs using a Python script and the rdflib library. Also, another intervention was necessary for the implementation of this thesis, this was the addition of '*Utility*'⁴⁸. The 'Utility' key was used to categorize 'items'⁴⁹ in a way that those 'items' can first be analyzed as a distinct category and later to conduct analyses which investigates the relations between items of different categories.

I first downloaded the specific .ttl file from the repository, which is the "Informatics-Skills.ttl"⁵⁰ then run the script "convert_saro.py" which converts the data from the turtle format into JSON with the same main structure but having also the 'Utility' as mentioned above, below there is the picture of one item from the resulting JSON:

⁴⁷ "In computing, Terse RDF Triple Language (Turtle) is a syntax and file format for expressing data in the Resource Description Framework (RDF) data model. Turtle syntax is similar to that of SPARQL, an RDF query language. It is a common data format for storing RDF data, along with N-Triples, JSON-LD and RDF/XML." [43]

⁴⁸ The 'Utility' was mentioned at Chapter 1, the Introduction.

⁴⁹ The SARO contents are called 'items'.

⁵⁰ There are other Ontology files available, each for a specified sector, those are: Health, Information & Technology, Business Administration, Legal, Science & Engineering and Teaching.

```
{
  "id": "python",
  "type": "Skill",
  "labels": [
    "python (computer programming)", "python", "Python"
  ],
  "isFrom": "ESCO",
  "isCoreTo": [
    "ICT"
  ],
  "utility": ["Programming Language"],
  "hasSkillType": "knowledge",
  "hasReuseLevel": "sector specific skills and competences"
},
```

Picture 17: saro.json

Core of the Method

A high-level overview of the methodology followed in this thesis is the following:

- A) **Mapping Phase:** The purpose of this phase is to take as a first input the mapping file '*saro.json*' (the file mentioned in the above chapter), second input dataset directly from the database and give as an output a CSV with the *saro_id*, the occurrence frequency of this item, the 'Utility' of the item and the list of job ids of the job posts that this item has been found in. Each category has its SARO_Mapping Jupyter notebook, so there are two notebooks on this implementation, one for the programming languages and one for the databases.
- B) **Comparison Phase:** This phase takes as an input the CSVs resulted from the "Mapping Phase" and calculates for each category the percentages of the respective items. The result is the percentages of each item in the dataset, more specifically the programming languages and databases.

Those were the necessary steps to get the first results of the Dictionary-based Entity Extraction Method, the comparison of programming languages and databases. The products of this process are also the input for the next processes, which are:

- C) **Combination Phase⁵¹:** In this phase the aim is to calculate the combinations of items of different categories, more specifically the combinations of a programming language item and a database item in the same job post. This is achieved by

⁵¹ The combination phase is conducted in the same notebook as the Comparison Phase, "languages_databases_results.ipynb"

iterating the list of each item and counting a match when the same id is in the list of an item of the other category.

- D) **Estimation of Roles for Each Language:** The purpose of this phase is to determine which roles are constituting of which items by conducting a Rule-Based Classification to the job title field. This is measured by analyzing the job title of each job post of the items list, categorizing each title into a specified group and then aggregating those categories resulting in the estimation of the roles for each item.

After implementing the methodology, the results were:

The First Results of the Method Implementation – Troubleshooting

The First Measurement and the Adjustment

At first, the total number of 'Programming Languages occurrences' (total) was all the **unique** ids; however, this was proved wrong because one item (Programming Language in this example) can occur in many job posts; hence the job posts should not be considered as a unique item holder.

Programming Languages

```
[103]: total_unique_ids
total=0
for i in range(0, languages.shape[0]):
    total=total+100*languages.iloc[i]['frequency']/len(lang_unique_ids)
    print(languages.iloc[i]['saro_id'],": " , 100*languages.iloc[i]['frequency']/len(lang_unique_ids))
print(total)

java : 36.875
python : 31.875
javascript : 17.375
go : 12.875
c : 6.625
c# : 5.625
typescript : 5.125
php : 4.875
Cplusplus : 3.375
swift : 2.25
kotlin : 2.25
scala : 2.0
ruby : 1.5
rust : 1.125
133.75
```

Picture 18: First result of the Dictionary-based Entity Extraction Method Implementation for programming languages, note that the summarized percentage is 133.75 %.

When calculated the percentage with the total being the number of occurrences of all items the percentages were right:

Programming Languages

```
[108]: total_unique_ids
total=0
for i in range(0, languages.shape[0]):
    total=total+languages.iloc[i]['frequency']
    print(languages.iloc[i]['saro_id'],": ", languages.iloc[i]['frequency'])
print(total)
test=0
for i in range(0, languages.shape[0]):
    print(languages.iloc[i]['saro_id'],": ", 100*languages.iloc[i]['frequency']/total)
```



```
java : 295
python : 255
javascript : 139
go : 103
c : 53
c# : 45
typescript : 41
php : 39
Cplusplus : 27
swift : 18
kotlin : 18
scala : 16
ruby : 12
rust : 9
1070
java : 27.570093457943926
python : 23.83177570093458
javascript : 12.990654205607477
go : 9.626168224299066
c : 4.953271028037383
c# : 4.205607476635514
typescript : 3.8317757009345796
php : 3.6448598130841123
Cplusplus : 2.5233644859813085
swift : 1.6822429906542056
kotlin : 1.6822429906542056
scala : 1.4953271028037383
ruby : 1.1214953271028036
rust : 0.8411214953271028
```

Picture 19: Programming Languages items calculated right in contrast to the Picture 18

Generally when calculating each category of items, the dataset is investigated only in this portion of the dataset that has that specific category and it does not compared against the whole dataset, this indicates a perspective of analyzing the results which for example says that: The IT sector has a percentage of jobs that programming languages are not within the requirements, so these job posts should not be considered a part of the overall job posts related to the programming languages when compared with each other. So, every category has its unique total, hence programming languages should be compared with each other and therefore the total must be the total occurrences of all programming language. The same is true for other technologies and skills like databases that will be investigated in this Thesis, and when multiple item categories are calculated together like programming languages and databases.

Errors of the Process

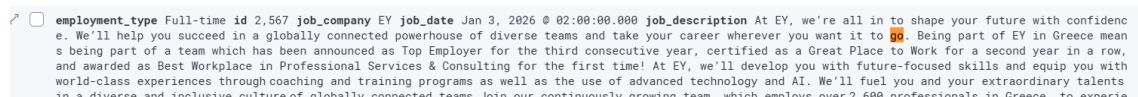
From this first implementation there were some errors which I oppose in this chapter:

‘Go’ Language

The calculation of ‘Go’ Language is not right, because in contrast with the measurements of the other languages, it has not a discrete difference from other words, more specifically: the algorithm counts the language ‘Go’ but also the verb ‘go’, those two have no difference from each other in the text, the only difference is semantic. I have confirmed this conflict by investigating a part of the ids from the list of the language ‘go’.

	saro_id	job_ids	frequency	utility
73	python	[2502, 2509, 12, 2516, 2524, 2548, 2550, 2553,...	498	[Programming Language]
71	java	[1, 2506, 2508, 2509, 12, 2515, 2516, 2519, 25...	313	[Programming Language]
51	javascript	[2498, 2506, 2508, 2515, 2519, 2531, 2532, 255...	298	[Programming Language]
70	go	[1, 2509, 2516, 2555, 2567, 2327, 2610, 233, 2...	180	[Programming Language]
93	typescript	[2506, 2508, 2515, 2519, 2532, 2552, 2567, 257...	161	[Programming Language]

Picture 20: Here the id 2567 can be seen in the ‘go’ results

 employment_type Full-time id 2,567 job_company EY job_date Jan 3, 2026 @ 02:00:00.000 job_description At EY, we're all in to shape your future with confidence. We'll help you succeed in a globally connected powerhouse of diverse teams and take your career wherever you want it to go. Being part of EY in Greece means being part of a team which has been announced as Top Employer for the third consecutive year, certified as a Great Place to Work for a second year in a row, and awarded as Best Workplace in Professional Services & Consulting for the first time! At EY, we'll develop you with future-focused skills and equip you with world-class experiences through coaching and training programs as well as the use of advanced technology and AI. We'll fuel you and your extraordinary talents in a diverse and inclusive culture of globally connected teams. Join our continuously growing team which employs over 7,600 professionals in Greece to experience

Picture 21: Here we can see the job post description, the ‘go’ item that was counted was wrong, because the semantic meaning of ‘go’ in this description is the verb.

Results From the Dictionary-Based Entity Extraction

Programming Languages

The final results of the Dictionary-Based Entity Extraction after corrections can be seen below:

Total number of all mentions of programming languages: 1972			
python	498	25.25%	
java	313	15.87%	
javascript	298	15.11%	
go	180	9.13%	
typescript	161	8.16%	
c	144	7.30%	
c#	114	5.78%	
CPlusPlus	92	4.67%	
php	57	2.89%	
kotlin	32	1.62%	
swift	24	1.22%	
scala	21	1.06%	
ruby	20	1.01%	
rust	18	0.91%	

So, let’s analyze the first results that we get from the main methodology:

1. Python has a very strong position on the sector of technology, it is a result that that appears logical from the aspect that Python can be used not only from Software Engineers and Developers but also from other roles for example, Data Engineer role is usually a crucial skill and it is a role that is in high demand, as we will see later. This result also aligns with the 2025 Stack Overflow survey which finds that Python's popularity has accelerated significantly in the past years, typical example of this is that it has increased 7 percent from the same survey in 2024.
2. The second language is 'Java', this makes sense for Greece. It is rank that can be confirmed from the fact that the Universities in Greece are mainly tutoring 'Java' along with 'C' as well, it also makes sense because it confirms a trend which empirically is noticeable with a quick look in the job portals from the past years.
3. In the third position is JavaScript but very close to java, this is also close with the survey of Stack Overflow where JavaScript is the first in popularity, however a part of this differentiation has to do with the content of the jobs in the dataset but also indicates that it is not dominant in Greek ICT field.
4. In 'Go' language the measurement is not right as mentioned in this chapter above, because the algorithm also counted the verb go, so for this implementation the go language will not be counted as valid, in future implementations this language can be counted only with semantic data analysis.
5. Typescript usually goes together with JavaScript, so this is an expected place.
6. At the sixth place is 'C' language, this makes sense especially if we consider the place of 'Go' language is not really the 4th but it is certain that it is below. Also, considering that if we combine the 3 'C' languages the percentage reaches 17.75% placing the 'C-family languages' at a certain second position.

Databases

Total number of all mentions of databases: 653		
postgresql	142	21.75%
sqlserver	135	20.67%
mysql	113	17.30%
redis	64	9.80%
databricks	58	8.88%
elasticsearch	30	4.59%
snowflake	27	4.13%
mariadb	22	3.37%
bigquery	18	2.76%
cassandra	12	1.84%

hive	7	1.07%
oraclesql	6	0.92%
dynamodb	6	0.92%
sqlite	5	0.77%
tsql	3	0.46%
solr	3	0.46%
sql azure	1	0.15%
mongodb	1	0.15%

1. From the algorithm results in databases, we see that the first database in demand on this dataset is PostgreSQL. This trend totally aligns with the survey of Stack Overflow where PostgreSQL is at the first place in popularity of all the databases.
2. The next is the ‘SQL Server’, there is a differentiation here from the survey, which possibly highlights that it is a distinction of the Greek IT sector, the fact that more companies are using ‘SQL Server’.
3. MySQL is third, makes sense, and aligns with the Stack Overflow survey.
4. The place Redis gets in the research aligns with the Stack Overflow survey which has a very similar position, and states that compared with the same survey conducted in 2024 Redis had an increase of 8%, a fact that is amplified by the language strong position.
5. The position Databricks have in the results is either a distinct feature of the dataset, or of the measurement, and it differentiates on a significant level from the Stack Overflow survey where its place is below the medium. If this trend is confirmed, we can say that it is a unique feature of the Greek IT sector.
6. Elasticsearch is not far from the place the stack overflow gives, it is an expected result.

Combinations

Here are the combinations of programming languages and databases, the combinations are derived from matching different items together, from the list of ids of each item. To put it more simply, if item A and item B are items of different categories and they are in the same job posts, then this is counted as a combination.

Total combinations are: 1018

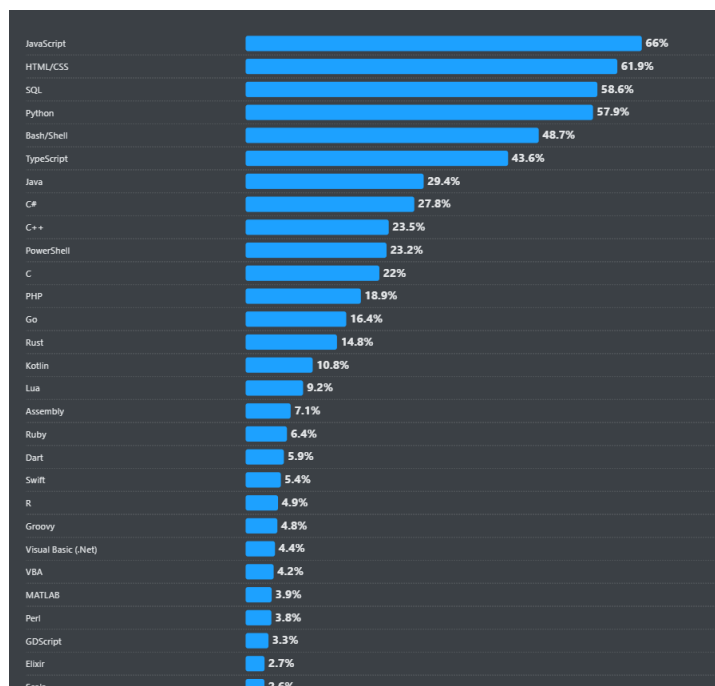
('postgresql', 'python') 6.58 %	('databricks', 'go') 0.39 %
('postgresql', 'java') 5.21 %	('databricks', 'c#') 0.39 %
('mysql', 'java') 4.91 %	('snowflake', 'java') 0.39 %
('databricks', 'python') 4.91 %	('DB2', 'java') 0.39 %

('sqlserver', 'c#') 4.03 %	('DB2', 'go') 0.39 %
('postgresql', 'javascript') 3.73 %	('postgresql', 'rust') 0.29 %
('mysql', 'javascript') 3.73 %	('mysql', 'CPlusPlus') 0.29 %
('sqlserver', 'python') 3.54 %	('mysql', 'ruby') 0.29 %
('mysql', 'python') 3.44 %	('redis', 'swift') 0.29 %
('sqlserver', 'javascript') 2.85 %	('databricks', 'c') 0.29 %
('sqlserver', 'java') 2.75 %	('elasticsearch', 'go') 0.29 %
('postgresql', 'typescript') 2.65 %	('elasticsearch', 'php') 0.29 %
('postgresql', 'go') 2.55 %	('oraclesql', 'typescript') 0.29 %
('redis', 'javascript') 2.46 %	('hive', 'scala') 0.29 %
('redis', 'python') 2.36 %	('postgresql', 'kotlin') 0.2 %
('redis', 'java') 2.36 %	('mysql', 'swift') 0.2 %
('mysql', 'go') 2.26 %	('mysql', 'scala') 0.2 %
('mysql', 'php') 2.26 %	('elasticsearch', 'typescript') 0.2 %
('snowflake', 'python') 1.87 %	('snowflake', 'javascript') 0.2 %
('postgresql', 'php') 1.47 %	('snowflake', 'c') 0.2 %
('sqlserver', 'go') 1.47 %	('cassandra', 'javascript') 0.2 %
('elasticsearch', 'javascript') 1.47 %	('cassandra', 'c#') 0.2 %
('postgresql', 'c#') 1.28 %	('cassandra', 'scala') 0.2 %
('postgresql', 'c') 1.18 %	('hive', 'java') 0.2 %
('mysql', 'c#') 0.98 %	('hive', 'c') 0.2 %
('mysql', 'typescript') 0.88 %	('dynamodb', 'typescript') 0.2 %
('redis', 'go') 0.88 %	('solr', 'java') 0.2 %
('redis', 'typescript') 0.88 %	('sqlserver', 'CPlusPlus') 0.1 %
('redis', 'c#') 0.88 %	('sqlserver', 'kotlin') 0.1 %
('elasticsearch', 'java') 0.88 %	('sqlserver', 'ruby') 0.1 %
('sqlserver', 'typescript') 0.79 %	('mysql', 'kotlin') 0.1 %
('mysql', 'c') 0.79 %	('mysql', 'rust') 0.1 %
('redis', 'ruby') 0.79 %	('redis', 'scala') 0.1 %
('databricks', 'scala') 0.69 %	('redis', 'rust') 0.1 %
('elasticsearch', 'ruby') 0.69 %	('elasticsearch', 'kotlin') 0.1 %
('postgresql', 'CPlusPlus') 0.59 %	('elasticsearch', 'swift') 0.1 %
('redis', 'c') 0.59 %	('snowflake', 'typescript') 0.1 %
('redis', 'CPlusPlus') 0.59 %	('snowflake', 'CPlusPlus') 0.1 %
('redis', 'php') 0.59 %	('cassandra', 'go') 0.1 %
('databricks', 'java') 0.59 %	('cassandra', 'CPlusPlus') 0.1 %
('snowflake', 'scala') 0.59 %	('oraclesql', 'python') 0.1 %
('cassandra', 'python') 0.59 %	('oraclesql', 'java') 0.1 %
('cassandra', 'java') 0.59 %	('hive', 'CPlusPlus') 0.1 %
('dynamodb', 'python') 0.59 %	('dynamodb', 'go') 0.1 %
('postgresql', 'scala') 0.49 %	('sqlite', 'python') 0.1 %
('elasticsearch', 'python') 0.49 %	('sqlite', 'java') 0.1 %

('snowflake', 'go') 0.49 %	('sqlite', 'javascript') 0.1 %
('DB2', 'python') 0.49 %	('sqlite', 'go') 0.1 %
('hive', 'python') 0.49 %	('sqlite', 'c') 0.1 %
('postgresql', 'ruby') 0.39 %	('sqlite', 'kotlin') 0.1 %
('sqlserver', 'c') 0.39 %	('sqlite', 'ruby') 0.1 %
('sqlserver', 'php') 0.39 %	('mongodb', 'java') 0.1 %
('redis', 'kotlin') 0.39 %	('mongodb', 'typescript') 0.1 %
('databricks', 'CPlusPlus') 0.39 %	('mongodb', 'kotlin') 0.1 %

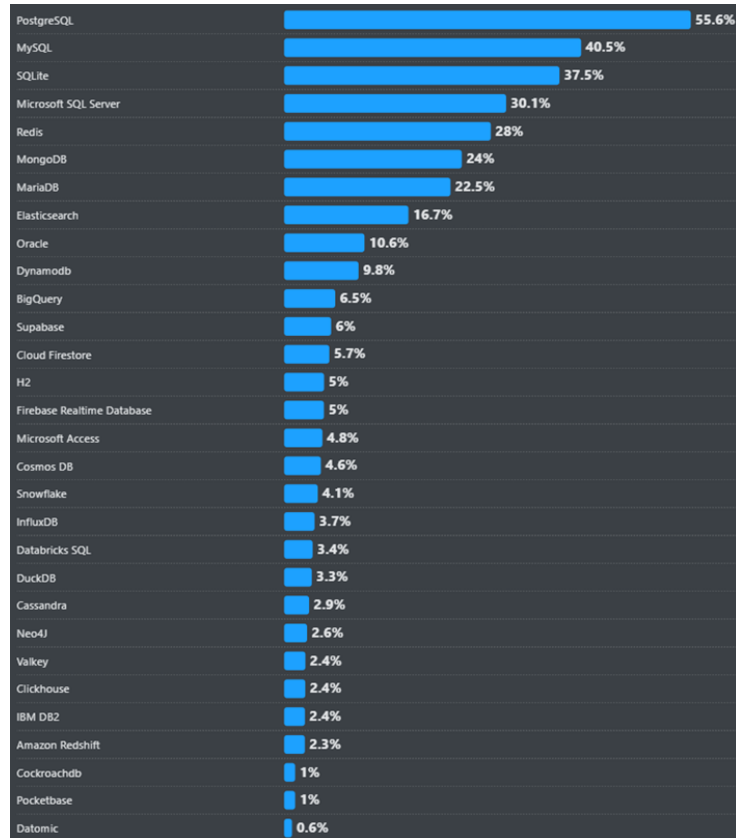
1. The dominant combinations are those expected based on the results of the respective categories
2. One interesting combination is ('databricks', 'python') 4.91 %. Databricks and Python are typically used for data-intensive, distributed, and analytics-driven systems which possibly indicate that a big part of the dataset consists of data engineer/analyst/scientist etc.

Below is the Programming Languages part of the Stack Overflow Survey:



Picture 22: Programming Languages part of the Stack Overflow Survey

Below is the Databases part of the Stack Overflow Survey:



Picture 23: Databases part of the Stack Overflow Survey

6.3.5 Analysis of The Roles Corresponding to Items

In this part, an analysis will be conducted to each of the Items⁵² to determine on which roles each occurs. This will be achieved by processing the title of the job posts so that every job post can be labeled to a category and then calculate which items are related to which role categories. The role category determination is conducted by matching specific string values in the job title using regex. Each group of string values corresponds to a specific 'Key' which is the role category, the format of this Key-Value categorization is in JSON, below is a part of this categorization file:

"Data & AI": "(data|machine learning|ml|ai|\\b|artificial intelligence|big data)",

Results of Roles Corresponding to the Dataset

These are the results of the categorization based on the job post title for the whole dataset:

--- Demand Distribution of The Dataset ---

⁵² With the term 'Items' meaning the SARO items searched, hence 'Java', 'Python', 'Postgres', 'MySQL' etc

Category	
Infrastructure & Support	25.79%
Other / Uncategorized	22.17%
Backend & General Eng	16.33%
Management & Architecture	10.08%
Data & AI	9.45%
Frontend	4.74%
ERP & Business Systems	3.83%
Full Stack	3.36%
QA & Testing	2.73%
Mobile	1.51%

Note that the category ‘Other / Uncategorized’ is second which means that this categorization needs to be updated, especially when implementing it to the whole dataset.

Results of Roles Corresponding to Items

The following are the results of the above-mentioned process:

--- Demand Distribution for: PYTHON --- Data & AI 38.15% Backend & General Eng 22.09% Infrastructure & Support 15.26% Management & Architecture 10.84% Other / Uncategorized 4.22% Full Stack 4.02% QA & Testing 2.81% Frontend 2.01% ERP & Business Systems 0.6%	--- Demand Distribution for: JAVA --- Backend & General Eng 33.55% Management & Architecture 14.7% Full Stack 9.9% Data & AI 8.95% Infrastructure & Support 8.63% QA & Testing 7.99% Frontend 6.71% Other / Uncategorized 4.47% Mobile 3.19% ERP & Business Systems 1.92%
--- Demand Distribution for: JAVASCRIPT --- Frontend 34.23% Backend & General Eng 23.15% Full Stack 17.11% QA & Testing 6.04% Management & Architecture 5.37% Infrastructure & Support 4.03% Data & AI 3.69% Other / Uncategorized 3.36%	--- Demand Distribution for: GO --- Backend & General Eng 21.67% Management & Architecture 18.89% Data & AI 14.44% Infrastructure & Support 12.78% Other / Uncategorized 12.22% Frontend 8.89% Full Stack 5.0% ERP & Business Systems 4.44%

ERP & Business Systems 2.01%	QA & Testing 1.11%
Mobile 1.01%	Mobile 0.56%
--- Demand Distribution for: TYPESCRIPT ---	--- Demand Distribution for: C ---
Frontend 35.4%	Backend & General Eng 28.47%
Management & Architecture 20.5%	Infrastructure & Support 23.61%
Full Stack 16.77%	Other / Uncategorized 12.5%
Backend & General Eng 16.15%	Data & AI 11.81%
QA & Testing 6.83%	Management & Architecture 8.33%
Data & AI 2.48%	QA & Testing 6.25%
Infrastructure & Support 1.24%	Frontend 3.47%
Other / Uncategorized 0.62%	Mobile 2.78%
	Full Stack 1.39%
	ERP & Business Systems 1.39%
--- Demand Distribution for: C# ---	--- Demand Distribution for: C++ ---
Backend & General Eng 55.26%	Backend & General Eng 56.52%
Full Stack 9.65%	Data & AI 17.39%
Management & Architecture 7.89%	Other / Uncategorized 8.7%
Data & AI 7.89%	Infrastructure & Support 7.61%
QA & Testing 7.89%	Management & Architecture 6.52%
Other / Uncategorized 6.14%	Mobile 2.17%
Frontend 3.51%	Full Stack 1.09%
Infrastructure & Support 1.75%	
--- Demand Distribution for: PHP ---	--- Demand Distribution for: KOTLIN ---
Backend & General Eng 36.84%	Mobile 46.88%
Frontend 21.05%	Backend & General Eng 15.62%
Full Stack 21.05%	Other / Uncategorized 9.38%
Infrastructure & Support 8.77%	Frontend 6.25%
Other / Uncategorized 7.02%	Data & AI 6.25%
Data & AI 1.75%	QA & Testing 6.25%
Management & Architecture 1.75%	Management & Architecture 6.25%
QA & Testing 1.75%	Infrastructure & Support 3.12%
--- Demand Distribution for: SWIFT ---	--- Demand Distribution for: SCALA ---
Mobile 54.17%	Data & AI 61.9%
Infrastructure & Support 12.5%	Management & Architecture 14.29%
Backend & General Eng 8.33%	Backend & General Eng 9.52%
Frontend 8.33%	QA & Testing 9.52%

Other / Uncategorized 8.33%	Infrastructure & Support 4.76%
Management & Architecture 4.17%	
QA & Testing 4.17%	
--- Demand Distribution for: RUBY ---	--- Demand Distribution for: RUST ---
Backend & General Eng 45.0%	Backend & General Eng 61.11%
Full Stack 15.0%	Full Stack 22.22%
Infrastructure & Support 15.0%	Management & Architecture 11.11%
Mobile 10.0%	QA & Testing 5.56%
Frontend 10.0%	
ERP & Business Systems 5.0%	

The results are placed with a descending order, from the most used programming language which is Python to the least used in the dataset which is Rust. The results seem to be grounded in reality if we look at the top findings regarding the quantity of the data:

- A) Python is used mainly for Data and AI.
- B) Java is the first programming language for Backend.
- C) JavaScript together with TypeScript are the first programming languages for Frontend.
- D) The C family languages are used mainly for Backend.
- E) Swift and Kotlin are used almost entirely for mobile.

Further Analysis of The Items in Elasticsearch

Those results can be further analyzed and investigated by using Elasticsearch, in the following pages we will see what specific characteristics can be found by searching the job posts that matched an item. This will be achieved technically by taking the ids of an item and searching it with a KQL query in Kibana, this outputs the respective job posts which then can be analyzed with field statistics and other tools. The item that will be investigated here will be 'Python':

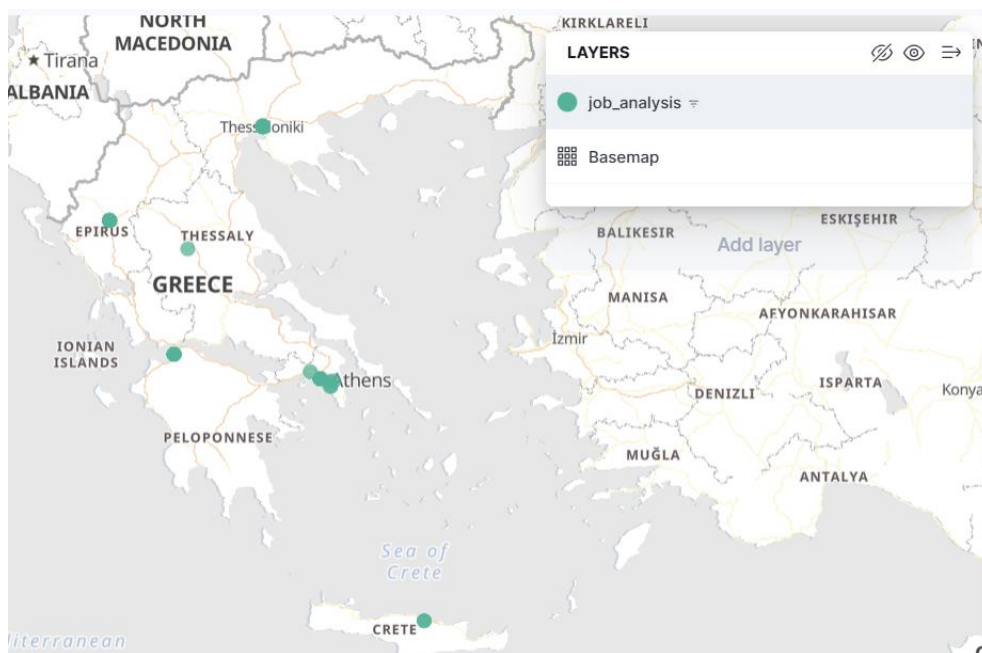
Map results: the reason why the *'and not workplace: "Greece"'* query was added is because some workplaces were either not indicated, or the workplace was 'Greece', in both cases the workplace is considered blank, hence those cases were excluded from the

map results:



Picture 24: KQL for the map results.

One of the first findings that we can get from Elasticsearch, since the regional normalization and geolocation have been configured is a map overview of the item that we are investigating, here is where job posts with the ‘Python’ item were found:



Picture 25: Locations where ‘Python’ item has been found

Field statistics: this feature gives statistics for each of the fields of the job posts, it is necessary to hit a KQL query to get field statistics, below is the query for investigating the ‘Python’ results. Note that in this query the ‘*and not workplace: "Greece"*’ is not present, this is because all other attributes are investigated except from the location.

This is the KQL query for all the results except from the map results:



Picture 26: investigating ‘Python’ results in Elasticsearch

Documents (498)		Field statistics				Actions
Type	Name	Documents (%)	Distinct values	Distributions		
employment_type	employment_type	498 (49.8%)	11			
employment_type.keyword	employment_type.keyword	498 (100%)	11	top 10 of 11 categories		
id	id	498 (100%)	498	2	3,189	
job_company	job_company	498 (49.8%)	193			
job_company.keyword	job_company.keyword	498 (100%)	193	top 10 of 193 categories		
job_date	job_date	490 (98.39%)	142			
job_description	job_description	498 (49.8%)	454			
job_id	job_id	498 (49.8%)	498			
job_id.keyword	job_id.keyword	498 (100%)	498	top 10 of 498 categories		
job_sector	job_sector	498 (49.8%)	73			
job_sector.keyword	job_sector.keyword	498 (100%)	73	top 10 of 73 categories		
job_title	job_title	498 (49.8%)	335			
job_title.keyword	job_title.keyword	498 (100%)	335	top 10 of 335 categories		
location	location	498 (100%)	12			
site	site	498 (100%)	4	4 categories		

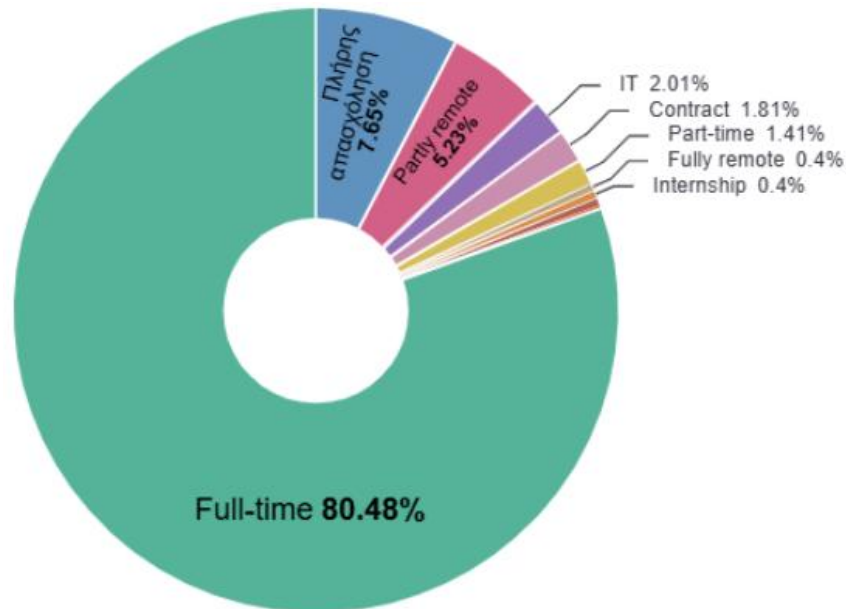
Picture 27: Field statistics of the ‘Python’ item

More specifically we can see and configure statistics of each column, in the below examples we will see some columns that give potentially valuable results.

- a) Employment Type:** This column has not yet been normalized and there are erroneous fields have not yet been corrected⁵³, however even with the given situation it seems that it is a field which can depict a tendency which indicates reality, given

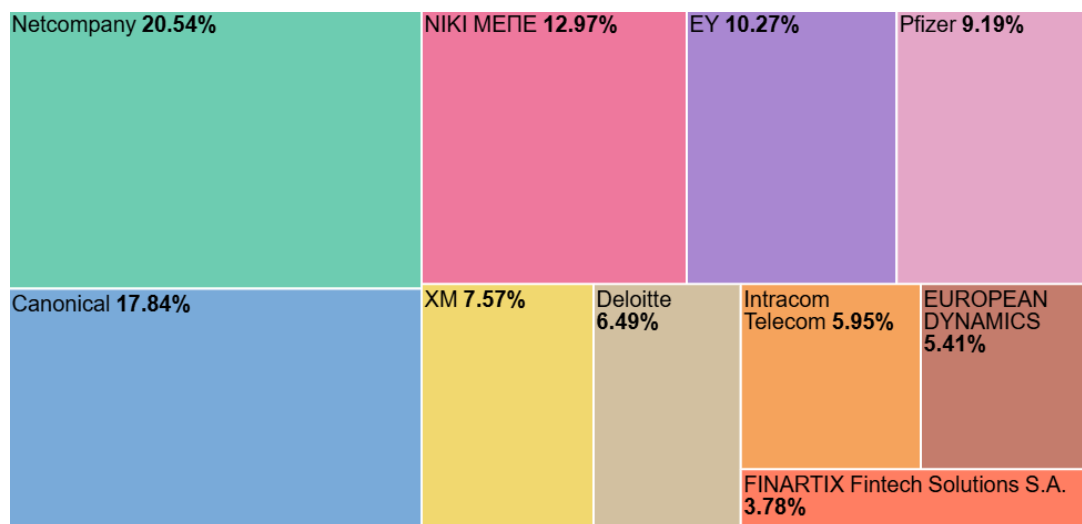
⁵³ In this example we can clearly see the ‘IT’ 2,19%.

that employment type is a field which is present in all job portal sites and it is considered a required field:



Picture 28: Employment Type of the item 'Python'

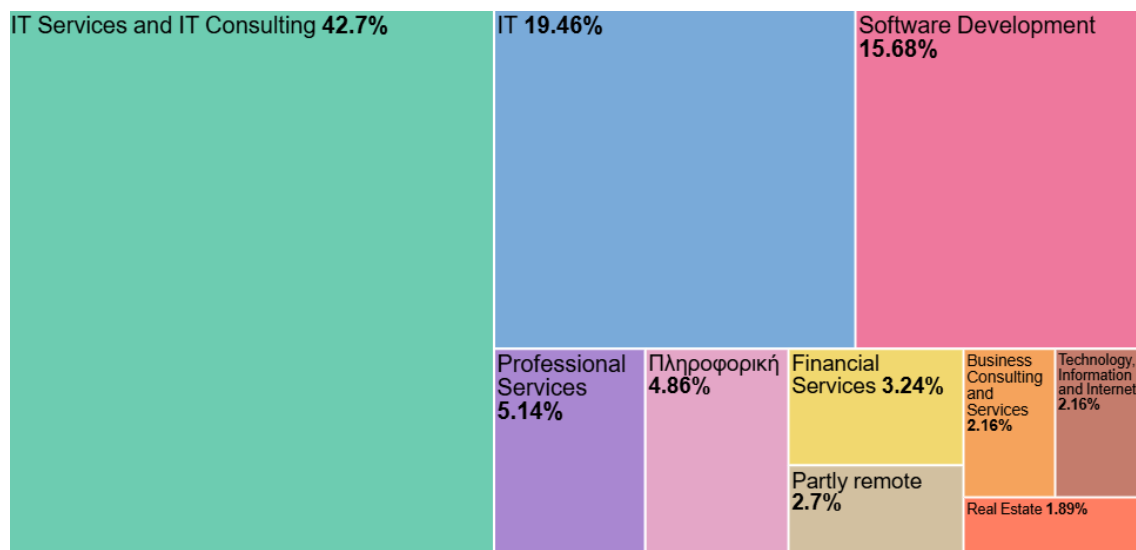
- b) **Job Company:** In this metric we see the percentages of the top 10 companies that are searching for employees and possess at any level of skill the 'item' we are investigating, in this example those are companies that are searching for 'Python':



Picture 29: Companies where in their job posts the item 'Python' exists.

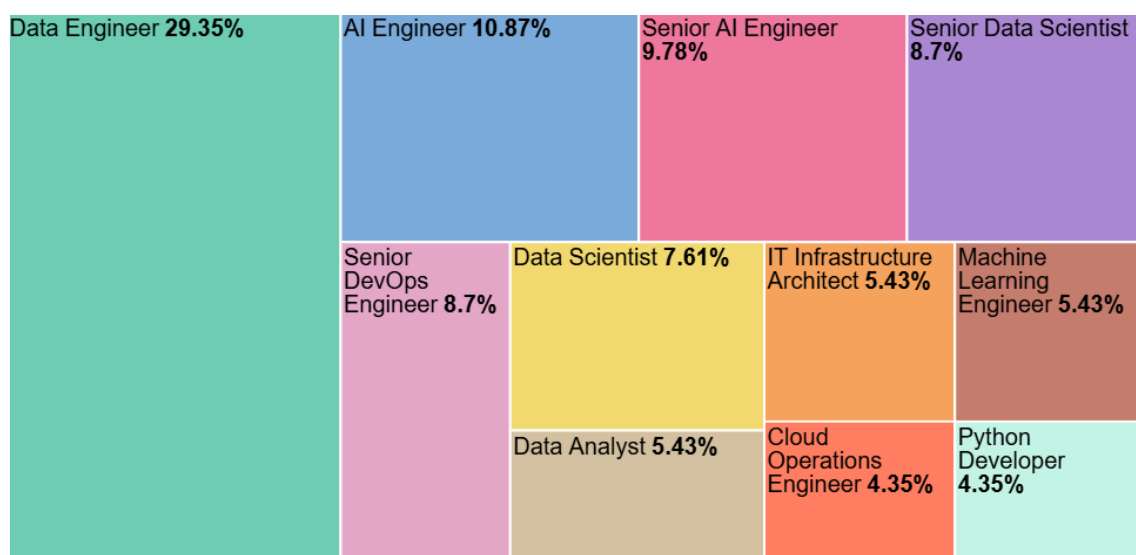
- c) **Job Sector:** This field does not give any valuable results for the dataset that is being analyzed, which are job posts from the IT sector. Even if it was a case

with more than one sector, it is a field that needs normalization and corrections⁵⁴ as mentioned in a previous chapter⁵⁵. However, it is a field that it can be useful for other implementations as it will be discussed in the "Conclusions" Chapter 7.



Picture 30: Job Sector where the item 'Python' exists.

- d) **Job Title:** Job Title field statistics give us a view of the roles that are related to a specific item. This overview does not give very high-value insight; however, this can validate to a degree that there is an alignment of the item that we are searching for with the job post we get as an output through the job title, below are the top 10 job titles that have in the description the item 'Python':



⁵⁴ In this example we see the error in 'Partly remote' 2,96% which is an employment type

⁵⁵ In the chapter "Core Metadata Fields" the subchapter Job Sector

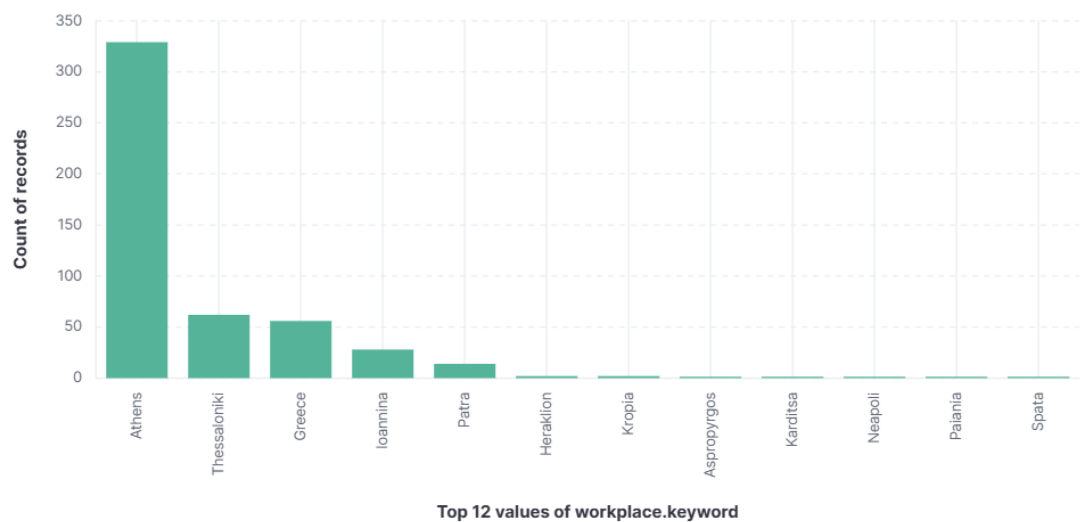
Picture 27: Job Titles from job posts where the item 'Python' exists.

- e) **Site:** Here we see the percentages of each site that posted the item that we search:



Picture 31: Sites where the item 'Python' exists.

- f) **Workplace:** Here we see the percentage of the workplace, essentially is the data of the map we viewed above, but on a XY plot where X axis is the workplace keywords and the Y axis is the count of job posts records:



Picture 32: Workplace where the item 'Python' exists.

Those were the fields that give us less or more valuable insights, the other fields do not offer any kind of usable information, e.g. id field because it is not information that indicates aspects of the data.

The above statistics can be conducted for any item, and of course for any result of the data analytics part if the ids are available. So, this can be done for any programming language, any database, their combinations and for any item we define and investigate in the dataset.

7 Conclusions

Results Critique and Commentary

From the results we can conclude that for the job posts dataset the methods of conducting data analysis with a pre-defined dictionary gave more concrete results than the TF-IDF algorithms. This discrepancy is primarily attributed to the linguistic characteristics of job descriptions, which are frequently dominated by verbose prose regarding corporate culture, company history, and internal policies rather than technical requirements or 'hard skills'.

In most instances, the specific mentions of technologies and tools occupy a disproportionately small percentage of the total text. Therefore, when applying TF-IDF to extract technological trends, the algorithm does not distinguish high-frequency 'noise' words from significant technical terms. This is evidenced by the TF-IDF output, where the statistically significant terms frequently consist of generic descriptive vocabulary rather than relevant domain-specific identifiers.

Consequently, a dictionary-based extraction method was adopted to facilitate the precise identification of pre-defined technical entities. By utilizing this approach, the investigation shifted from general frequency analysis to targeted data localization. This allowed for an accurate assessment of the distribution and relative density of specific technologies across the dataset, providing a robust comparative analysis of their prevalence.

One notable disadvantage of this method is that we can only locate the terms that we define, and this can result in losing, for example, new technologies and new skills emerging in the sector that we are investigating. This is why this method, if followed for a prolonged period, should be frequently updated with anything new that has occurred.

Another issue that applies to some of the items is the fact that some words can have many meanings, for example 'go' language with 'go' verb. As has been analyzed in the chapter above, this can critically affect the measurement of a specific item, making it unusable for data analysis. This issue could possibly be promptly resolved with the utilization of an AI API integrated within our pipeline, however not resolving it is degrading the quality of the results this method offers.

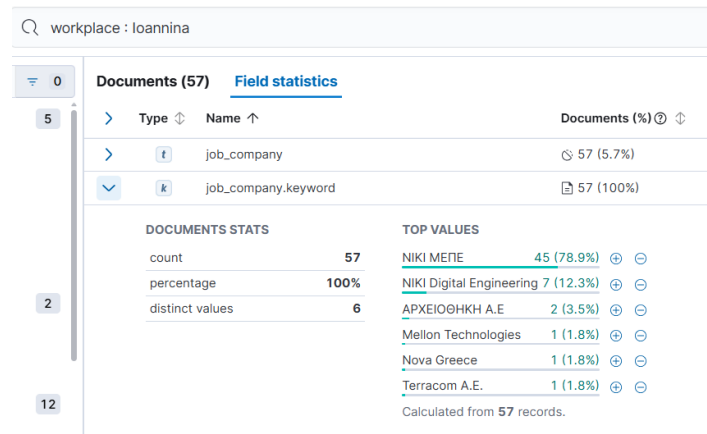
Future Implementation

One future implementation would be to work with both methods, by first mapping the dictionary into the dataset and then utilizing methods like TF-IDF to investigate the relations of those items and their occurrences and co-occurrences in the dataset. It is important to indicate that the dictionary derived from SARO in its current form has 1307 items, so after updating it with the recent technologies, tools and skills it would be a lexicon of the job descriptions that we can filter the whole description and ignore all the other words that cause the noise. The process could be then to implement the TF-IDF for example or any other data analysis algorithm and possibly get core structure of each job post.

This implementation of analytics was conducted with data from the IT field, however this could be done for other sectors and to more job portals possibly mapping the whole job market of a specific area or country. This would give us the potential of being able to see in real time what professions have an increasing demand and what are the skills those professions are consisting of, also this could give skill groups relating with professions and observe trends in technologies and tools companies are using.

One perspective that can be useful for future implementations is the map and location data, observing job portal data based on the map could inform us about areas where there are emerging companies and jobs in a specific sector. One paradigm of that, based on the thesis results is the tech jobs in city of Ioannina which is the 24th city of Greece [29] based on population, making it a city that tech jobs are not expected. The results for python and other programming languages⁵⁶ indicate a high demand for employees with high specialization. nevertheless, when further investigating this trend, it proved to be one company which had posted the same job posts on different sites which boosted the workplace ‘Ioannina’ in the Elasticsearch field statistics. This example highlights the fact that on the one hand it is possible to detect such trends, however, those trends need examination to be determined if indeed they are legitimate trends or if it is due to biased data. This conclusion can be seen below where a query with Kibana conducted for the workplace ‘Ioannina’:

⁵⁶ The results for the rest of the programming languages are not in this document; those data are external.



Picture 33: The example of increased job posts from workplace Ioannina, where it is mainly due to one company

Another novelty the Elasticsearch map feature gives, is that we can search for anything in the dataset, any skill, technology, tool etc, essentially any string in relation to any field in the whole dataset. This is not something which is available on the public and it could be useful for many different stakeholders. Querying such a dataset of a specified area or country is feasible as this thesis proved, and the utilities this ability could have are still unknown since this is something new, certainly for Greece and as far as the research goes, for other counties as well. Similar utilities like this thesis examines, have the O*NET in the US, which is a database for professions and their skills. Still, searching the skills related to locations is something that I have not seen existing even in that platform.

To scale up to a bigger implementation one necessary technical optimization is the automation of the scraping process, hence programming web scraping to be conducted after specific periods of time, possibly with something like a cron job⁵⁷. Especially for the LinkedIn scraper, programming it to automate the process of job post ids management and extraction. Generally minimising the manual processes in the existing implementation. This implementation was conducted with a dataset extracted with job posts for about 5 months and with relative scarce scraping sessions, an implementation that the extraction conducted for longer period, with increased frequency and aiming to more sites and job portals, multiplies the potential possibilities of insights discovery.

⁵⁷ A scheduled job is known as a cron job. Although typically used to automate system maintenance and administration it can be used to automate any task. [45]

A promising avenue for future research involves the further optimization of Elasticsearch. This could be achieved by migrating the dataset to a cloud-based infrastructure and leveraging native AI capabilities. Such an integration would enable users to query the dataset directly using natural language.

Concluding, in future implementations the role category determination of job posts through the job titles method should be examined. More specifically, the accuracy of the existing categories, how well those categories depict the roles in the IT industry and if the job posts are indeed successfully categorised, hence a validation of the method. Also, another development that could be considered is adding more role categories.

Bibliography

- [1] W. A. F. M. A. A. M. A. O. Z. A. W. L. W. Ioannis Karakatsanisa, "Data mining approach to monitoring the requirements of the job market: A case study," 2017.
- [2] S. S. C. M. S. a. D. Elisa Margareth Sibarani, "Ontology-guided Job Market Demand Analysis," 2017.
- [3] "Scrapy," [Online]. Available: <https://www.scrapy.org>.
- [4] <https://notebooklm.google.com/notebook/84907e8a-f683-48a4-932c-48a52e21ae45>, "NotebookLM," [Online].
- [5] EU, "esco.ec.europa.eu/en/about-esco," EU, [Online]. Available: <https://esco.ec.europa.eu/en/about-esco>.
- [6] wikipedia, "wikipedia.org/wiki/Occupational_Information_Network," wikipedia, [Online]. Available: https://en.wikipedia.org/wiki/Occupational_Information_Network.
- [7] C. S. Jyldyz Djumalieva, An Open and Data-driven Taxonomy of Skills Extracted from Online Job Adverts, London: Economic Statistics Centre of Excellence, 2018.

- [8] M. R. B. M. Raymond Blanch Mbah, Discovering Job Market Trends with Text Analytics, St. Paul, MN 55105, USA: International Conference on Information Technology, 2017.
- [9] A. A. N. M. L. A. MARIA PAPOUTSOGLU, Extracting Knowledge From On-Line Sources for Software Engineering Labor Market: A Mapping Study, IEEE Access, 2019.
- [10] A. H. A. S. Asta Bäck, Text Mining on Job Advertisement Data: Systematic Process for Detecting Artificial Intelligence Related Jobs, Finland: Technical Research Centre of Finland, 2021.
- [11] H. U. M. C. E. S. T. K. W. L. W. M. A. O. Armin Alibasic, Evaluation of the trends in jobs and skill-sets using data analytics: a case study, Abu Dhabi, UAE: Journal of Big Data, 2022.
- [12] J. Z. Y. C. Taizhi Lva, Job Analysis System Based on Spark Platform, Jiangsu Maritime Institute, Nanjing 211170, China: International Conference on Financial Management, Education and Social Science, 2020.
- [13] T. B. M. M. Ž. K. Mirjana Pejic-Bacha, Text mining of industry 4.0 job advertisements, Zagreb, Croatia: International Journal of Information Management, 2020.
- [14] D. S. D. S. LEKHA R. NAIR, Streaming Twitter Data Analysis Using Spark For Effective Job Search, Dubai, UAE: Journal of Theoretical and Applied Information Technology, 2015.
- [15] N. Z. E. M. K. C. G. Giannis Tzimas, From Data to Insight: Transforming Online Job Postings into Labor-Market Intelligence, MDPI, 2024.
- [16] P. A. Jeonghyun Kim, Research using job advertisements: A methodological assessment, Library & Information Science Research, 2016.
- [17] kaggle, "kaggle.com," kaggle, [Online]. Available: <https://www.kaggle.com>.
- [18] AndresHG, "kaggle.com," kaggle, [Online]. Available: <https://www.kaggle.com/code/andreshg/nlp-glove-bert-tf-idf-lstm-explained>.

- [19] <https://stackoverflow.com>, "stackoverflow," Stackoverflow, 2025. [Online]. Available: <https://survey.stackoverflow.co/2025/technology>.
- [20] scrapy, "<https://docs.scrapy.org/>," scrapy, [Online]. Available: <https://docs.scrapy.org/en/latest/topics/autothrottle.html>.
- [21] wikipedia, "[wikipedia.org/wiki/LinkedIn](https://en.wikipedia.org/wiki/LinkedIn)," wikipedia, [Online]. Available: <https://en.wikipedia.org/wiki/LinkedIn>.
- [22] psycpg, "psycpg.org/psycpg3/docs/basic/install.html," psycpg, [Online]. Available: <https://www.psycpg.org/psycpg3/docs/basic/install.html>.
- [23] wikipedia, "<https://wikipedia.org/>," Wikipedia, [Online]. Available: <https://en.wikipedia.org/wiki/Elasticsearch>.
- [24] elastic.co, "<https://www.elastic.co/>," Elastic, [Online]. Available: <https://www.elastic.co/elasticsearch>.
- [25] elastic.co, "<https://www.elastic.co/>," Elastic, [Online]. Available: <https://www.elastic.co/docs/manage-data/data-store/aliases>.
- [26] elastic.co, "<https://www.elastic.co/>," Elastic, [Online]. Available: <https://www.elastic.co/docs/manage-data/data-store/mapping>.
- [27] elastic.co, "[elastic.co/docs/explore-analyze/query-filter/languages/kql](https://www.elastic.co/docs/explore-analyze/query-filter/languages/kql)," elastic.co, [Online]. Available: <https://www.elastic.co/docs/explore-analyze/query-filter/languages/kql>.
- [28] elisasibarani, "github.com/elisasibarani/SARO," elisasibarani, [Online]. Available: <https://github.com/elisasibarani/SARO>.
- [29] wikipedia, "[wikipedia.org/wiki/List_of_cities_and_towns_in_Greece](https://en.wikipedia.org/wiki/List_of_cities_and_towns_in_Greece)," wikipedia.org, [Online]. Available: https://en.wikipedia.org/wiki/List_of_cities_and_towns_in_Greece.
- [30] "spark.apache.org/," Apache, [Online]. Available: <https://spark.apache.org/docs/latest/sql-data-sources-jdbc.html>.
- [31] python.org, "[python.org](https://docs.python.org/3/library/venv.html)," Python, [Online]. Available: <https://docs.python.org/3/library/venv.html>.

- [32] Wikipedia, "Wikipedia," Wikipedia.org, [Online]. Available: [https://en.wikipedia.org/wiki/Pip_\(package_manager\)](https://en.wikipedia.org/wiki/Pip_(package_manager)).
- [33] Wikipedia, "Wikipedia," Wikipedia.org, [Online]. Available: https://en.wikipedia.org/wiki/User-Agent_header.
- [34] wikipedia, "Cardinality_(SQL_statements)," wikipedia, [Online]. Available: [https://en.wikipedia.org/wiki/Cardinality_\(SQL_statements\)](https://en.wikipedia.org/wiki/Cardinality_(SQL_statements)).
- [35] T. H. a. A. Reuter, "Principles of transaction-oriented database recovery," ACM, 1983.
- [36] wikipedia.org, "wikipedia.org/wiki/ACID," wikipedia, [Online]. Available: <https://en.wikipedia.org/wiki/ACID>.
- [37] wikipedia, "wikipedia.org/wiki/CAP_theorem," wikipedia, [Online]. Available: https://en.wikipedia.org/wiki/CAP_theorem.
- [38] wikipedia, "wikipedia.org/wiki/Unstructured_data," wikipedia, [Online]. Available: https://en.wikipedia.org/wiki/Unstructured_data.
- [39] freeCodeCamp.org, "freeCodeCamp.org," freeCodeCamp.org, [Online]. Available: https://www.youtube.com/watch?v=a4HBKEda_F8.
- [40] elastic.co, "elastic.co/docs/explore-analyze/discover/show-field-statistics," elastic.co, [Online]. Available: <https://www.elastic.co/docs/explore-analyze/discover/show-field-statistics>.
- [41] psycopg, "psycopg.org/psycopg3/docs/basic/install.html," psycopg, [Online]. Available: <https://www.psycopg.org/psycopg3/docs/basic/install.html>.
- [42] python, "docs.python.org/3/tutorial/errors.html#handling-exceptions," python.org, [Online]. Available: <https://docs.python.org/3/tutorial/errors.html#handling-exceptions>.
- [43] wikipedia, "wikipedia.org/wiki/Turtle_(syntax)," wikipedia.org, [Online]. Available: [https://en.wikipedia.org/wiki/Turtle_\(syntax\)](https://en.wikipedia.org/wiki/Turtle_(syntax)).
- [44] wikipedia, "wikipedia.org/wiki/Document_Object_Model," wikipedia.org, [Online]. Available: https://en.wikipedia.org/wiki/Document_Object_Model.

[45] wikipedia, "wikipedia.org/wiki/Cron," wikipedia, [Online]. Available:
<https://en.wikipedia.org/wiki/Cron>.

